



Konzept: Cloud-natives, verteiltes Dokumentenmanagementsystem (DMS)

Version: 0.18 (Entwurf)

Status: Konzeptphase

1. Zielbild & Leitprinzipien

Das System ist ein revisionssicheres, hochgradig konfigurierbares Dokumentenmanagementsystem, das als verteilte Microservice-Architektur betrieben wird. Zentrale Leitprinzipien:

1. **Erweiterbarkeit durch "Dazustellen"**: Neue Funktionalität (Connectoren, Verarbeitungsschritte, UI-Module) wird durch Hinzufügen neuer Plugin-/Service-Instanzen realisiert, nicht durch Eingriffe in bestehenden Code. Jede neue Instanz meldet sich selbstständig beim Registry Service an und wird vom Plugin Orchestration Service (3.8) automatisch auf der verfügbaren Serverlandschaft platziert.
2. **Separation of Data Flow und Control Flow**: Der Registry Service steuert *wer wen erreichen darf und ob er das darf* (Control Flow inkl. Lizenzstatus), während der eigentliche Dokumenten-/Datenverkehr direkt zwischen den beteiligten Services läuft.
3. **Konfiguration statt Code**: Objektdefinitionen, Bedingungen, Berechtigungsstrukturen, Workflows sind über JSON-Konfiguration importier- und exportierbar – Systeme sollen sich klonen bzw. deren Konfiguration übertragen lassen.
4. **Auditierbarkeit als Architekturprinzip, nicht als Feature**: Jede schreibende und lesende Aktion erzeugt ein unveränderliches Audit-Ereignis. Das ist kein nachträglich aufgesetztes Logging, sondern strukturell in jedem Service verankert.
5. **Schema-getrennte Shared Database**: Eine gemeinsame Datenbankinstanz (operativ einfacher: Backups, Skalierung, Betrieb), aber strikte Schema-Trennung pro Service – kein Service liest oder schreibt außerhalb seines eigenen Schemas.
6. **Funktionale Vollständigkeit ggü. etablierten Marktlösungen, mit echter horizontaler Skalierung als Differenzierung**: Das System soll im Kernfunktionsumfang mit etablierten Produkten wie nscale oder VIS-Suite mithalten (siehe Abschnitt 12 für den detaillierten

Marktvergleich). An Stellen, an denen deren Architektur strukturell begrenzt ist – etwa eine nicht durchgängig horizontal skalierbare Kernarchitektur bei klassischen EIM-Plattformen wie nscale –, soll das hier beschriebene Konzept durch konsequente zustandslose Skalierbarkeit jedes Services (in Verbindung mit dem Plugin Orchestration Service, 3.8) einen echten Vorteil bieten, statt Skalierung nachträglich aufzusetzen. Dieselbe Zustandslosigkeit ist zugleich die Grundlage für unterbrechungsfreie Rolling Updates (10.5) – ein Betriebsvorteil, den eine nachträglich skalierte Architektur nur schwer nachbilden kann.

1a. Backend-Technologiewahl (Python-first, polyglott-fähig)

Für den Start wird **Python** so weit wie möglich als Implementierungssprache für die Backend-Services eingesetzt. Das ist mit dem Architekturprinzip aus 3.5/3.2 bewusst vereinbar: Da alle Services ausschließlich über APIs miteinander kommunizieren (kein gemeinsamer In-Process-Code, keine sprachspezifische Kopplung), lässt sich jede einzelne Komponente später verlustfrei durch eine schnellere/andere Sprache ersetzen, sobald sich das für einen konkreten Service als nötig erweist (z. B. ein besonders durchsatzkritischer Rendering- oder Storage-Backend-Service in Go oder Rust) – **Python ist der pragmatische Startpunkt, keine dauerhafte Festlegung des gesamten Systems.**

Empfohlener Python-Stack

- **Web-/API-Framework:** FastAPI (async-first, native OpenAPI-Schema-Generierung – lässt sich direkt für die Plugin-/Connector-Manifeste aus 3.3/3.8 nutzen, Pydantic v2 für Validierung).
- **Datenbankzugriff:** SQLAlchemy 2.0 im Async-Modus mit asyncpg als Postgres-Treiber – aktueller Produktionsstandard im Python-Ökosystem für genau diese Kombination.
- **Event-Bus-Anbindung:** `nats-py` (offizieller asyncio-fähiger NATS-Client) für die Werkseinstellung NATS JetStream (3.4); für die Kafka-Alternative wahlweise `confluent-kafka-python` (C-Bindings, sehr performant) oder `aiokafka` (rein asyncio-nativ, passt konsistenter zum übrigen async-Stack).
- **Auth-Anbindung:** `Authlib` bzw. `python-jose` zur Validierung der von Keycloak ausgestellten OIDC-/JWT-Tokens in jedem Service (zustandslose Prüfung ohne Rückfrage

beim Auth Service, siehe 4.4); `python-keycloak` für administrative Interaktion mit Keycloak (Nutzer-/Gruppenabgleich) aus dem Auth Service heraus.

- **Storage-Backend-Anbindung** (konkretisiert 3.6): `aioboto3` / `boto3` für S3-kompatible Backends (funktioniert gegen AWS S3, MinIO, Ceph RGW und weitere S3-kompatible Provider mit derselben Bibliothek - passend zur Anforderung, mehrere S3-Provider parallel anzusprechen), `azure-storage-blob` für Azure Blob Storage, für das NFS-Backend regulärer Python-Dateisystemzugriff (`os` / `pathlib`) ergänzt um plattformspezifisches File-Locking (`fcntl` unter Linux) - hier ist besondere Sorgfalt bei NFS-Locking-Semantik nötig (NFSv4.1+ empfohlen, siehe auch 3.6).
 - **Query-Konsolen-Sprachimplementierung** (konkretisiert 6.1): Da der Abfrage-Dialekt eng an psql angelehnt ist und der Wunsch besteht, einen bestehenden Parser zu erweitern statt neu zu schreiben, bietet sich **pglast** an - ein aktiv gepflegtes Python-Modul, das den echten PostgreSQL-Parser (über `libpg_query`) als vollständigen, bearbeitbaren AST in Python verfügbar macht, inklusive Visitor-/Printer-Infrastruktur zur gezielten Erweiterung um die DMS-spezifischen Zusatzkonstrukte (Trace-/Hierarchie-Operatoren). Damit wird real PostgreSQL-Grammatik geparkt statt einer Nachbildung, was Kompatibilität und Wartungsaufwand deutlich verbessert.
 - **BPMN-Engine** (konkretisiert 7.1): **SpiffWorkflow** - eine seit über zehn Jahren gepflegte, reine Python-Workflow-/BPMN-Bibliothek mit explizitem Support für genau die hier benötigten BPMN-Elemente: Pools/Lanes (Abteilungsdarstellung), Multi-Instanz-Tasks, Sub-Workflows, Timer-/Boundary-Events (Basis für die SLA-Zeitüberwachung je Schritt, 7.1) sowie Signal-/Message-Events (Basis für die föderierte Prozessübergabe über den Federation Hub, 7.4). Für die grafische Modellierung existiert mit `bpmn-js-spiffworkflow` bereits eine passende Erweiterung des verbreiteten BPMN.js-Editors, die sich in den Process Designer (7.1/8) integrieren lässt, statt einen eigenen BPMN-Editor von Grund auf zu entwickeln.
 - **OCR-Engine** (konkretisiert 3.9): **PaddleOCR** (Apache-2.0) als Standard - deutlich robuster als klassisches Tesseract bei Tabellen, mehrspaltigen Layouts und mehrsprachigen Dokumenten; **Tesseract** (`pytesseract`) als ressourcenschonende Alternative für einfache Fälle über dasselbe Plugin-Interface.
 - **Signatur-Anbindung** (konkretisiert 3.10): **pyHanko** für die PDF-seitige PAdES-Signaturerzeugung/-einbettung/-prüfung (alle Baseline-Profile B-B/B-T/B-LT/B-LTA, PKCS#11 für Hardware-Token/HSM) - übernimmt die technische PDF-Arbeit, während die eigentliche Vertrauensdienstleistung beim angebundenen externen QTSP verbleibt.
-

2. Domänenmodell

2.1 Kernentitäten

Entität	Beschreibung
Dokument	Versionierte Datei mit Metadaten, Objekttyp, Status, Zugriffsrechten - alle Versionen bleiben dauerhaft erhalten und abrufbar (siehe 2.1a); optional mit Löschdatum/-frist und Kennzeichnung zur physischen Zwangslöschung (siehe 5.2a)
Ordner	Hierarchischer Container, selbst mit Metadaten/Objekttyp, trägt Berechtigungsvererbung
Umlaufmappe	Vorgangsbezogener Container mit Referenzen auf alle zu einem Vorgang gehörenden Dokumente - kein eigener Dateiinhalt, sondern Referenzstruktur (siehe 2.3); ebenfalls mit optionalem Löschdatum/-frist versehbar (siehe 5.2a)
Ersatzdarstellung	Alternative, aus einem Originaldokument abgeleitete Repräsentation für Barrierefreiheit/Ausfallsicherheit (z. B. .txt aus .docx, Transkript aus Video, siehe 2.4)
Prozessspezifische Bearbeitungskopie	Eigenständig abgelegte, für einen konkreten Vorgang angepasste Kopie eines Dokuments (z. B. Schwärzung für Akteneinsicht), getrennt von der reinen Versionierung (siehe 2.3)
Elektronische Signatur	Kryptografischer Nachweis (SES/AES/QES nach eIDAS) zu einer konkreten Dokumentversion, inkl. Validierungsstatus (siehe 3.10)
OCR-Textlayer	Durchsuchbare Texterkennung zu einem gescannten/bildbasierten Dokument, verknüpft mit der jeweiligen Version (siehe 3.9)
Objekttyp (Custom Object Definition)	Definiert Attribute, Pflichtfelder, Validierungsregeln für Dokumente/ Ordner
Bedingung/Regel (Constraint)	Regelwerk, das bei Erstellung/Änderung geprüft wird (z. B. "Attribut X muss gesetzt sein", "Name muss Muster Y erfüllen")

Entität	Beschreibung
BPMN-Prozess	Ausführbares oder importiertes BPMN-2.0-Modell, das Dokumente/Umlaufmappen durch Bearbeitungsschritte führt (siehe 7.1)
Berechtigungsrolle	RBAC-Rolle, ordner-/objektypbezogen zuweisbar
Nutzer/Gruppe	Lokal oder aus AD synchronisiert
Audit-Ereignis	Unveränderlicher Log-Eintrag zu jeder relevanten Aktion
Lizenz	Definiert Umfang/Limits eines Mandanten oder einer Installation

2.1a Versionierung (Klarstellung)

Alle Versionen eines Dokuments bleiben dauerhaft erhalten und über die gesamte Aufbewahrungsdauer hinweg abrufbar - kein Überschreiben, kein automatisches Verwerfen älterer Stände (Grundprinzip aus 4.2/5.1). Das gilt unverändert auch für Ersatzdarstellungen (2.4) und prozessspezifische Bearbeitungskopien (2.3): Beide sind eigenständige, ebenfalls versionierte Objekte, keine flüchtigen Zwischenstände.

2.2 Objekttypen & Custom Constraints

Objekttypen werden als Schema definiert (ähnlich JSON-Schema-Prinzip):

```

{
  "objectType": "Rechnung",
  "appliesTo": "document",
  "allowedParentTypes": ["Rechnungsordner", "Projektordner"],
  "attributes": [
    { "name": "Rechnungsnummer", "type": "string", "required": true, "pattern": "^RE-\\d{"
    { "name": "Betrag", "type": "decimal", "required": true },
    { "name": "Kostenstelle", "type": "string", "required": false }
  ],
  "namingConstraints": {
    "mustContain": ["Rechnungsnummer"],
    "pattern": "{Rechnungsnummer}_{Datum}"
  },
  "conditions": [
    { "if": "Betrag > 10000", "then": "require:Kostenstelle" }
  ]
}

```

Die Regel-Engine (siehe 4.5) wertet solche Bedingungen bei Erstellung, Änderung und Statusübergängen aus. Dasselbe Prinzip gilt für Ordner-Objektypen (z. B. "dieser Ordnertyp erfordert immer ein Attribut 'Projektnummer'"). `allowedParentTypes` ist optional (fehlt es, gilt der Typ als überall platzierbar wie bisher) - siehe 2.2a für die erzwungene Hierarchie und 2.2b für die layoutbezogenen Attribute (`icon` , Formular-Layout).

2.2a Erzwungene Objekt-Hierarchie & visuelle Klassifizierung

Damit sich eine vom Betreiber gewünschte Ablagestruktur nicht nur durch Konvention, sondern **erzwungen** durchsetzen lässt, kann jeder Objektyp (Ordner- wie Dokumentklassen) über `allowedParentTypes` (2.2) eine oder mehrere zulässige übergeordnete Ordnerklassen benennen:

- **Mehrfachangabe erlaubt** (ODER-Verknüpfung): eine Dokument-/Ordnerklasse kann mehrere zulässige Elternklassen benennen, z. B. weil sie fachlich unter zwei verschiedenen Oberordnertypen sinnvoll vorkommen kann.
- **Root als zulässiges Ziel**: die spezielle Angabe `"$ROOT"` erlaubt die Platzierung direkt unterhalb der Wurzel (analog zu bestehenden Root-Sonderfällen wie der `"root"` -Resource im Berechtigungsmodell, 4.1). Ein typisches Beispiel aus der Praxis:

`meinTopLevelOrd` erlaubt nur `"$ROOT"`, `meinSecondLevelOrd` erlaubt nur `meinTopLevelOrd`, `meinDoc` erlaubt nur `meinSecondLevelOrd` - so entsteht eine erzwungene, mehrstufige Ablagehierarchie statt einer frei begehbaren Ordnerstruktur.

- **Durchsetzung:** die Prüfung erfolgt durch dieselbe Regel-/Constraint-Engine (4.5), die auch die übrigen Objekttyp-Bedingungen auswertet - bei Anlage *und* beim Verschieben eines Ordners/Dokuments wird geprüft, ob die Klasse des Ziel-Elternordners in `allowedParentTypes` der zu platzierenden Klasse enthalten ist. Ein Verstoß wird wie jeder andere Constraint-Verstoß abgelehnt (2.2), nicht nur als Warnung angezeigt.
- **Ex-/Importierbarkeit:** `allowedParentTypes` ist ein gewöhnliches Feld der Objekttyp-Definition und damit automatisch Teil des ohnehin für Objekttypen vorgesehenen JSON-Ex-/Imports (7.3) - keine separate Konfigurationsebene nötig.

Zusätzlich kann jede **Ordnerklasse** ein `icon` (2.2b) tragen, das im User-UI-Explorer vor dem Namen jeder Instanz dieser Klasse angezeigt wird (8) - so lassen sich Ordner unterschiedlicher Klassen auf einen Blick unterscheiden, ohne den vollen Objekttyp-Namen lesen zu müssen.

2.2b Formular-Layouts (Anzeige, Suche, Upload)

Jeder Objekttyp trägt zusätzlich zu seinen Attributen (2.2) ein **Formular-Layout**: eine JSON-Repräsentation, die beschreibt, wie die Attribute dieses Typs in der User-UI angeordnet und beschriftet werden. Das Layout wird nicht von Hand als JSON/Text verfasst, sondern **generativ über einen GUI-Editor im Admin-UI erzeugt** (8): Die administrierende Person wählt aus, welche Attribute zur Klasse gehören, vergibt je Attribut einen sprechenden Anzeigenamen (unabhängig vom internen technischen Namen) und legt fest, welche Attribute Pflichtfelder sind - aus diesen Angaben leitet das System automatisch ein sinnvolles Standardlayout ("Smart Layout") ab, das serverseitig als JSON gespeichert wird.

- **Ein Layout, drei Verwendungszwecke:** dasselbe Layout-Format steuert (a) die Metadaten-Anzeige/-Bearbeitung eines Dokuments/Ordners dieser Klasse, (b) die Filterfelder der Suchmaske (3.7) für diesen Objekttyp, und (c) die Eingabefelder der Uploadmaske. Jeder der drei Verwendungszwecke kann - über denselben **Layout-Designer** im Admin-UI - vom generierten Standardlayout abweichend angepasst werden (z. B. ein Attribut, das im Uploaddialog Pflicht ist, aber in der Suchmaske optional bleibt), ohne die zugrunde liegende Objekttyp-Definition selbst zu ändern.
- **Grid-Modell:** ein Layout gliedert sich in Zeilen und/oder Spalten - Attribute können nebeneinander (spaltenweise) oder untereinander (zeilenweise) gruppiert werden, frei

durch die administrierende Person im Layout-Designer festgelegt.

- **Responsives Verhalten:** unterhalb einer konfigurierbaren Fensterbreite wird ein mehrspaltiges Layout unabhängig von seiner Autorenkonfiguration immer einspaltig (nur noch zeilenweise gestapelt) dargestellt - Lesbarkeit auf schmalen Fenstern/kleinen Bildschirmen hat Vorrang vor der ursprünglich entworfenen Spaltenaufteilung.
- Wie die Objekttyp-Definition selbst (2.2) ist auch das Layout Teil der Admin-seitigen Konfiguration - Endnutzer der User-UI sehen nur das Ergebnis, nicht den Editor.

2.3 Umlaufmappen & prozessspezifische Bearbeitungskopien

Umlaufmappen

- Eine Umlaufmappe bündelt **Referenzen** (keine eigenen Kopien) auf alle Dokumente, die zu einem bestimmten Vorgang gehören - vergleichbar einer digitalen Akte/Laufmappe, die durch einen BPMN-Prozess (7.1) durch verschiedene Abteilungen bzw. Instanzen wandert.
- Eine Umlaufmappe ist selbst ein eigenständiges, RBAC- und constraint-fähiges Objekt (2.2) mit eigenem Objekttyp, eigenen Attributen (z. B. Vorgangsnummer, zuständige Abteilung, Fälligkeit) und eigenem Lebenszyklus über die Workflow Engine (7.1).
- **Referenzverhalten während der aktiven Bearbeitung vs. bei Abschluss** (zweistufiges Modell):
 - **Während der Vorgang läuft**, referenziert die Umlaufmappe je enthaltenem Dokument **dynamisch die jeweils aktuellste Version** - wird ein referenziertes Dokument im Laufe der Bearbeitung weiter geändert (neue Version, 2.1a), zeigt die Umlaufmappe automatisch auf diesen neuesten Stand, ohne dass die Referenz manuell nachgeführt werden muss.
 - **Beim Abschluss der Umlaufmappe** (Erreichen des Endzustands im zugehörigen BPMN-Prozess, 7.1) wird die Referenzstruktur **fixiert**: Für jedes zu diesem Zeitpunkt enthaltene Dokument wird die dann aktuellste Version als **Abschluss-Snapshot** festgeschrieben - die Umlaufmappe zeigt ab diesem Moment nicht mehr dynamisch auf "die jeweils neueste Version", sondern konkret auf genau die Versionsstände, die zum Abschlusszeitpunkt vorlagen. Spätere Änderungen an den referenzierten Originaldokumenten (sofern diese weiterhin eigenständig bearbeitet werden) verändern den fixierten Abschluss-Snapshot nicht mehr.
 - Dieser Abschluss-Snapshot wird zusammen mit der Umlaufmappe automatisch archiviert (siehe Archivierbarkeit unten) und ist die Grundlage dafür, den **tatsächlichen Bearbeitungsstand zum Vorgangsabschluss** später lückenlos

nachvollziehen zu können - unabhängig davon, wie sich die referenzierten Dokumente danach weiterentwickeln.

- **Archivierbarkeit:** Umlaufmappen unterliegen denselben Regeln wie Dokumente - Aussonderung/Langzeitarchivierung (5.6) schließt Umlaufmappen samt ihrer (nach Abschluss fixierten) Referenzstruktur ein, nicht nur die referenzierten Einzeldokumente. Der Abschluss-Snapshot stellt dabei sicher, dass die referenzierten Dokumentversionen zum Zeitpunkt des Vorgangsabschlusses eindeutig und dauerhaft nachvollziehbar bleiben.
- Beim Löschen/Aussondern eines referenzierten Einzeldokuments bleibt die Referenz in der Umlaufmappe nachvollziehbar bestehen (Verweis auf "war vorhanden, jetzt archiviert/ausgesondert"), statt kommentarlos zu verschwinden - wichtig für die spätere Rekonstruierbarkeit eines Vorgangs.

Prozessspezifische Bearbeitungskopien

- Für Fälle, in denen ein Dokument im Rahmen eines konkreten Vorgangs in einer angepassten Form benötigt wird - klassisches Beispiel: eine **Schwärzung** für die Akteneinsicht einer Bürgerin/eines Bürgers, weil das Originaldokument personenbezogene Daten Dritter enthält -, wird diese angepasste Fassung **nicht als neue Version des Originaldokuments** behandelt, sondern als eigenständige, gesondert hinterlegte Bearbeitungskopie mit Verweis auf das Ursprungsdokument (inkl. konkreter Ausgangsversion) und den auslösenden Vorgang/die auslösende Umlaufmappe.
- Diese Trennung ist bewusst: Das Originaldokument bleibt unverändert und vollständig einsehbar (für berechtigte Personen), während die Bearbeitungskopie ausschließlich für den jeweiligen Vorgangszweck sichtbar/verwendbar ist - eigene Zugriffsrechte, eigener Objekttyp (z. B. "Bearbeitungskopie - Schwärzung"), eigene Aufbewahrungsregel (kann z. B. an die Dauer des auslösenden Vorgangs gekoppelt sein, statt an die Aufbewahrungsfrist des Originals).
- Erstellung, Änderung und Löschung solcher Bearbeitungskopien werden wie jede andere Dokumentaktion vollständig auditiert (5.3) und sind ebenfalls versioniert (2.1a).

2.4 Ersatzdarstellungen (Barrierefreiheit & Ausfallsicherheit)

Zusätzlich zur regulären Vorschau-/Renderfunktion (Rendering/Preview Service, 3.7) unterstützt das System **Ersatzdarstellungen**: eigenständig abgelegte, aus einem Originaldokument abgeleitete alternative Repräsentationen, die zwei Zwecken dienen:

- **Ausfallsicherheit/Teilausfall-Zugriff:** z. B. eine .txt-Extraktion aus einem .docx, falls die

Word-Verarbeitung/-Vorschau zeitweise ausfällt, oder eine .pdf-Konvertierung aus einer .pptx, falls die PowerPoint-Verarbeitung Probleme macht - Nutzer behalten so auch bei einem Teilausfall einzelner Rendering-Komponenten Zugriff auf den Inhalt in einem alternativen, robusteren Format.

- **Barrierefreiheit:** z. B. eine Transkription eines Videos für gehörlose Nutzer, oder allgemein alternative Textrepräsentationen für Screenreader-Zugänglichkeit.

Technisch:

- Ersatzdarstellungen werden vom Rendering/Preview Service (3.7) erzeugt (regelbasiert/konfigurierbar, welche Zielformate je Quellformat generiert werden - z. B. "bei .docx immer .txt-Ersatzdarstellung vorhalten", "bei Video immer Transkript erzeugen, falls ein Transkriptions-Plugin verfügbar ist") und als eigenständige, mit dem Original verknüpfte Objekte abgelegt - nicht als flüchtiger, bei Bedarf neu erzeugter Cache-Eintrag, da die Verfügbarkeit gerade im Ausfallszenario nicht von der Funktionsfähigkeit des ursprünglichen Renderers abhängen soll.
- Wie beim Rendering allgemein ist die konkrete Erzeugung **plugin-/service-basiert** (3.3/3.8) - neue Zielformate oder neue Transkriptions-/Konvertierungsfähigkeiten (z. B. ein neues Video-Transkriptions-Plugin) lassen sich nach dem "Dazustellen"-Prinzip ergänzen, ohne den Kern zu ändern.
- Ersatzdarstellungen sind wie das Original versioniert (2.1a) und unterliegen denselben Berechtigungen wie das zugrunde liegende Dokument, sofern nicht abweichend konfiguriert.
- Barrierefreiheits-Ersatzdarstellungen sind vom Konzept her von den prozessspezifischen Bearbeitungskopien (2.3) klar zu unterscheiden: Erstere sind eine vollständige, originalgetreue alternative Repräsentation desselben Inhalts für alle Nutzer, Letztere sind eine bewusst inhaltlich veränderte (z. B. geschwärzte) Fassung für einen bestimmten Vorgangszweck.

2.5 Bereichsstruktur: Dokument-Root und Sonderbereiche

Die eigentliche Aktenablage folgt einer über Objekttypen (2.2) und die erzwungene Hierarchie (2.2a) frei konfigurierbaren Struktur - ein typisches, aber nicht fest vorgegebenes Beispiel aus der Verwaltungspraxis: **Akte** (Dokument-Objekttyp) liegt in einem **Aktenplan** (Ordner-Objekttyp), dieser liegt in einer **Abteilung** (Ordner-Objekttyp), diese liegt direkt unter der **Dokument-Root** ("\$R00T" , 2.2a). Jede Installation kann diese Klassen/Ebenen beliebig anders

benennen, mehr oder weniger Ebenen vorsehen oder mehrere parallele Wurzelklassen zulassen
 - das Beispiel dient nur der Veranschaulichung des Mechanismus aus 2.2a.

Neben dieser konfigurierbaren Dokument-Root sieht das System auf derselben strukturellen Ebene weitere, vom System selbst bereitgestellte **Sonderbereiche** vor - vergleichbar den Sonderordnern eines E-Mail-Programms (Posteingang, Papierkorb, ...), fachlich aber auf die Aktenverwaltung zugeschnitten. Ein Sonderbereich ist kein gewöhnlicher, frei anlegbarer Ordner: er existiert genau einmal je Installation (bzw. je Mandant, 3a), sein Inhalt entsteht überwiegend durch Systemvorgänge statt durch manuelles Anlegen, und er trägt ein eigenes, vom übrigen RBAC-Modell (4.1) unabhängig konfigurierbares Zugriffsregime.

Sonderbereich	Zweck	Zugriff	Bezug
Posteingang/ Postausgang	Externe Korrespondenz (z. B. E-Mails von/an andere Stellen, Übergaben von/an andere Installationen), die noch keinem Vorgang zugeordnet ist, landet hier statt direkt in einem Aktenplan.	Nur eine dedizierte Poststelle-Rolle sieht/ bearbeitet den ungesichteten Zulauf; erst nach Sichtung/Zuordnung (manuell oder über einen BPMN-Eingangsschritt, 7.1) wandert ein Element in den Aktenplan oder in eine neu/bestehend zugeordnete Umlaufmappe (2.3).	3.3 (Connector-Prinzip für den technischen Empfang, z. B. E-Mail-Connector), 7.1 (Zuordnungs-Workflow), 7.4 (ein föderierter Ein-/Ausgang von einer anderen Installation ist konzeptionell derselbe Fall)
Papierkorb	Sammelstelle für von regulären Nutzern zur	Reguläre Nutzer können hinein verschieben	4.6, 5.2

Sonderbereich	Zweck	Zugriff	Bezug
	Löschung markierte Dokumente/ Umlaufmappen (Soft-Delete, 5.2) - keine sofortige Löschung.	(markieren), aber nicht endgültig daraus löschen. Endgültiges Löschen ist einer eigenen, abgespeckten Admin-Rolle Löschadministration vorbehalten (Beispiel für eine domänengetrennte Admin-Rolle, 4.6).	
Verschlussachen-Papierkorb	Fachlich identisch zum regulären Papierkorb, aber strukturell getrennt für Dokumente, die als Verschlussache eingestuft sind (Kennzeichnung über einen Objekttyp/ein Attribut, 2.2 - keine eigene Klassifizierungsstufen-Systematik ist Teil dieses Konzepts).	Eigene, engere Löschadministration (Verschlussachen)-Rolle, personell nicht zwingend deckungsgleich mit der regulären Löschadministration - Trennung, damit reguläres Löschadmin-Personal nicht automatisch Einsicht in als Verschlussache eingestufte Inhalte erhält.	4.6
Persönlicher Papierkorb	Personenbezogene Sicht: zeigt jeder Nutzerin/jedem Nutzer ausschließlich die von ihr/ihm selbst zur Löschung markierten Elemente samt Status ("wartet auf Löschadministration"), unabhängig vom vollständigen, für	Jeder Nutzer sieht nur die eigenen Einträge - kein Einblick in fremde Löschemarkierungen.	5.2

Sonderbereich	Zweck	Zugriff	Bezug
	Löschadmins sichtbaren Papierkorb-Bestand.		
Quarantäne	Dokumente, die den verpflichtenden Virensan vor Freigabe (10.3) nicht bestanden haben, landen hier statt im regulären Bestand.	Eine eigene, eng begrenzte Rolle darf einen Quarantäne-Fall einsehen, endgültig löschen oder - nach Klärung eines Fehlalarms - freigeben; die Freigabe ist ein auditiertes Vorgang (5.3), keine stille Wiederherstellung.	10.3
Kontakte	Verzeichnis zum Auffinden anderer Mitarbeitender - primär innerhalb der eigenen Installation (auf Basis der bestehenden Nutzerverwaltung, 4.4), optional installationsübergreifend , sofern eine Föderation (7.4) konfiguriert und für Kontaktsuche freigegeben ist. Bewusst zu unterscheiden vom Adressbuch des Federation Hub (7.4), das <i>Installationen</i> als Teilnehmer verzeichnet, nicht einzelne Personen - eine installationsübergreifende	Sichtbarkeit je Installation konfigurierbar (z. B. nur eigene Abteilung, ganze Installation, föderierte Installationen).	4.4, 7.4

Sonderbereich	Zweck	Zugriff	Bezug
	Kontaktsuche ist eine eigene, auf dem Federation Hub aufsetzende Fähigkeit, keine automatische Nebenwirkung der bestehenden Föderationsfunktion.		
Aussonderung	Bereits ausgesonderte, aber noch innerhalb der Übergangsfrist befindliche Akten/Umlaufmappen (5.6) bleiben hier durchsuch-/einsehbar, statt ausschließlich über Metadaten-/Audit-Trail-Verweise rekonstruierbar zu sein. Nach Ablauf der Übergangsfrist verschwindet ein Element aus diesem Bereich (reguläre Löschung im aktiven System, 5.6), bleibt aber im Langzeitarchiv/Audit-Trail weiterhin nachvollziehbar.	Zugriff i. d. R. auf dieselben Rollen beschränkt, die auch vor der Aussonderung Zugriff hatten, zusätzlich ggf. eine dedizierte Archiv-/Registratur-Rolle.	5.6, 7.2
Vorlagen	Wiederverwendbare Struktur-Vorlagen, z. B. ein vollständiger Aktenplan-Rohbau (Ordnerklassen samt üblicher Unterebenen) für eine neu	Anlegen/Ändern von Vorlagen ist administrativ geschützt (z. B. domänengetrennte Admin-Rolle, 4.6); Anwenden einer Vorlage	7.3

Sonderbereich	Zweck	Zugriff	Bezug
	einzurichtende Abteilung. Technisch dieselbe Grundlage wie der ohnehin vorhandene JSON-Struktur-Ex-/Import (7.3): eine Vorlage ist ein benannter, im Vorlagen-Bereich abgelegter Struktur-Export, ihre Anwendung ein gezielter Struktur-Import unterhalb eines gewählten Zielorts.	kann breiter freigegeben sein (z. B. für jede Person, die ohnehin eine neue Abteilung/einen neuen Aktenplan anlegen darf).	

Weitere Sonderbereiche sind über dasselbe Prinzip jederzeit ergänzbar ("Dazustellen", 1.) - ein naheliegender Kandidat aus der klassischen Schriftgutverwaltung, der bewusst (noch) nicht Teil dieser Konzeptversion ist, aber sich ohne strukturellen Bruch ergänzen ließe: eine **Wiedervorlage** (Dokumente/Vorgänge, die zu einem späteren, konkreten Zeitpunkt automatisch wieder zur Bearbeitung vorgelegt werden - technisch dieselbe Timer-Infrastruktur wie die Löscherinnerung, 5.2a, und die SLA-Zeitüberwachung, 7.1, nur fachlich anders verwendet).

Abgrenzung zur schmalen iconbasierten Navigationsleiste in der User-UI (8): Der dort erwähnte "Aufgaben/Postkorb"-Punkt bezeichnet die **Workflow-Aufgabenliste** (offene Freigaben/ Bearbeitungsschritte aus 7.1) und ist **nicht** identisch mit dem hier beschriebenen Posteingang/ Postausgang (externe, noch nicht zugeordnete Korrespondenz) - beide Begriffe klingen ähnlich, meinen aber unterschiedliche Dinge und werden in UI/Terminologie klar getrennt benannt.

3. Architektur

3.1 Grundprinzip: Shared Database, getrennte Schemata

- Eine physische Datenbankinstanz (z. B. PostgreSQL-Cluster) für einfacheren Betrieb

(Backup/Restore, Skalierung, Monitoring an einer Stelle).

- Jeder Microservice besitzt **exakt ein eigenes Schema**, auf das ausschließlich er lese-/schreibend zugreift. Andere Services fragen Daten ausschließlich über die API des jeweiligen Owner-Service ab, nie per Direktzugriff auf ein fremdes Schema.
- Schema-Migrationen sind Teil der Service-Pipeline des jeweiligen Owners; kein Service ändert das Schema eines anderen.
- Für serviceübergreifende Auswertungen (Reporting, Suche) wird nicht direkt gejoint, sondern über Events/Read-Modelle (siehe 3.4) oder einen dedizierten Reporting-Service mit eigener, replizierter Sicht gearbeitet.
- Technische Durchsetzung: DB-Nutzer pro Service mit GRANT ausschließlich auf das eigene Schema; das verhindert versehentliche Kopplung auch bei Bugs.

3.2 Registry Service – Herzstück der Verteilung

Der Registry Service übernimmt zwei eng verzahnte Aufgaben:

a) Service Discovery (Control Flow)

- Jeder Service registriert sich beim Start mit: Service-Typ, Instanz-ID, Version, unterstützte Capabilities, Health-Endpoint, Erreichbarkeitsadresse.
- Heartbeat/Health-Check in konfigurierbarem Intervall; fällt ein Service aus, wird er aus der aktiven Routingtabelle entfernt.
- Andere Services fragen die Registry, welche Instanz eines benötigten Service-Typs aktuell erreichbar ist (Basis für Client-seitiges Load Balancing oder Verweis auf einen internen Gateway).

b) Lizenzvermittlung

- Der Registry Service selbst führt keine Lizenzprüfung durch, sondern fragt den dedizierten License Service ab und reicht das Ergebnis an registrierende Services weiter.
- Bei der Registrierung erhält jeder Service eine Antwort mit Lizenzstatus für seine Komponente: licensed, demo, unlicensed.
- Der License Service publiziert Änderungen (z. B. Lizenz läuft ab, Limit überschritten) als Event; Registry aktualisiert die Auskunft, betroffene Services reagieren (z. B. Wechsel in Demo-Modus) ohne Neustart.
- Damit ist die Lizenzlogik selbst auch nur ein Service - neue Lizenzmodelle lassen sich einführen, ohne die Registry anzufassen.

3.3 Connector-Architektur (CMIS, WebDAV, Custom)

- Jeder Connector ist ein eigenständiger Service mit einheitlichem Connector-Interface (Capability-Beschreibung: lesen, schreiben, Metadaten, Locking, Versionierung - je nach Zielsystem unterschiedlich unterstützt).
- Registrierung analog zu allen anderen Services über die Registry - inkl. Lizenzprüfung (z. B. CMIS-Connector als eigene lizenzierbare Komponente, siehe Abschnitt 9).
- Ein Connector-SDK (Referenzimplementierung + Interface-Definition) ermöglicht es, weitere/kundenspezifische Connectoren zu bauen, ohne den Core zu verändern - das ist der technische Kern des "Dazustellen"-Versprechens.
- Versionierung des Connector-Protokolls (z. B. via API-Version im Registrierungs-Handshake), damit alte Connectoren nicht bei Core-Updates brechen.

3.4 Event-Bus für Auditierbarkeit und Entkopplung

Ergänzend zur Shared DB wird ein Event-Bus eingeführt:

- Jede relevante Aktion (Dokument erstellt, gelesen, geändert, gelöscht, Berechtigung geändert, Workflow-Schritt durchlaufen, Login etc.) erzeugt ein Event.
- Der Audit-Service konsumiert alle Events und schreibt sie unveränderlich (append-only, ggf. mit kryptografischer Verkettung/Hash-Chain zur Manipulationssicherheit) in sein eigenes Schema.
- Andere Services (Such-Index, Benachrichtigungsdienst, Reporting) können denselben Event-Strom konsumieren, ohne den Audit-Service oder andere Owner-Schemata direkt anzufragen - das reduziert Kopplung, obwohl die Basis eine Shared DB ist.
- Vorteil: Die Auditierbarkeit ist strukturell durch die Architektur erzwungen (kein Event = keine Aktion), nicht von der Disziplin einzelner Entwickler abhängig.

Konkrete Technologiewahl: Event-Bus als austauschbares Backend, analog zur Storage-Abstraktion (3.6)

- Statt sich früh auf eine einzige Event-Bus-Technologie festzulegen, wird der Event-Bus - konsistent mit dem Plugin-/Austauschprinzip des restlichen Systems - hinter einer eigenen Abstraktionsschicht betrieben, sodass die konkrete Technologie je Installation gewählt werden kann.
- **Werkseinstellung: NATS JetStream.** Passt am besten zum Modell vieler vollständig isolierter Einzelinstallationen (3a): Betrieb als einzelnes, leichtgewichtiges Binary ohne die

Cluster-/Betriebslast eines vollwertigen Kafka-Setups, gut geeignet für Request-Reply-Muster zwischen Services zusätzlich zum reinen Event-Streaming, und ressourcenschonend genug, um es pro Installation (statt zentral geteilt) zu betreiben.

- **Empfohlene Alternative für große Einzelinstallationen: Apache Kafka.** Sinnvoll, sobald eine einzelne Installation sehr hohen Durchsatz, sehr lange/dauerhafte Aufbewahrung des Event-Stroms (über die reguläre Audit-Aufbewahrung hinaus als technisches Replay-Log) oder tiefergehende Ökosystem-Anbindung (z. B. Kafka Connect für nachgelagerte Auswertungswerkzeuge) benötigt - auf Kosten des höheren Betriebsaufwands (produktiv mehrere Broker-Knoten, mehr Tuning-Aufwand).
- Beide Optionen sind über dasselbe interne Event-Publishing-/Konsum-Interface ansprechbar - Services selbst kennen nur "publiziere Event X" bzw. "konsumiere Events vom Typ Y", nicht die konkrete Bus-Technologie darunter.

3.5 API-Gateway / BFF-Schicht

- Zentrales API-Gateway für Authentifizierung (Token-Validierung), Rate Limiting, Routing zu Backend-Services (unter Nutzung der Registry).
- Pro Web-UI (User, Admin, ggf. weitere) ein schlankes Backend-for-Frontend, das UI-spezifische Aggregation übernimmt, statt die UI direkt viele Microservices ansprechen zu lassen.

3.6 Storage-Abstraktionsschicht

Die eigentlichen Dateiinhalte werden nicht in der relationalen Shared DB gehalten (dort nur Metadaten, Referenzen, Prüfsummen), sondern über einen eigenen **Storage Service** verwaltet, der als Abstraktionsschicht vor unterschiedlichen physischen Speicherbackends liegt.

Backend-Anbindung als Plugin-Prinzip

- Analog zur Connector-Architektur (3.3) ist jedes Speicherbackend (S3-kompatibel, NFS, Azure Blob, lokales Filesystem, künftig weitere) ein austauschbares **Storage-Backend-Plugin** mit einheitlichem Interface (schreiben, lesen, löschen, Existenzprüfung, Prüfsumme).
- Neue Backends lassen sich nach demselben "Dazustellen"-Prinzip ergänzen, ohne den Storage Service selbst anzufassen.

Redundante/parallele Speicherung

- Pro Mandant/Instanz konfigurierbar: ein oder mehrere Backends gleichzeitig als **Storage-Ziel-Set** (z. B. 2× S3 unterschiedlicher Provider + 1× NFS).
- **Beliebig viele gleichartige Backend-Instanzen:** das Ziel-Set ist keine feste Zuordnung "ein Backend-Typ = ein Ziel" - derselbe Backend-Typ kann mehrfach unabhängig konfiguriert im Ziel-Set auftauchen (z. B. 2× S3 gegen unterschiedliche Buckets/Provider zusätzlich zu 1× NFS-Mount), mit jeweils eigenen Zugangsdaten/Mount-Parametern/ Identitäts-Datei (siehe Datenträger-Wechsel-Sensibilität unten). Die Obergrenze der Anzahl ist rein betrieblich (sinnvolle Konfiguration liegt beim Betreiber), nicht architektonisch begrenzt.
- Schreibstrategie konfigurierbar (je Objekttyp/Ordnerbereich, nicht nur global):
 - **Synchron mit Quorum:** Schreiben gilt erst als erfolgreich, wenn eine konfigurierbare Mindestanzahl der Ziele bestätigt hat (z. B. 2 von 3) - stärkere Konsistenzgarantie, höhere Schreiblatenz. **Werkseinstellung für Bereiche mit hohem Schutzbedarf/ Integritätsanforderung** (z. B. revisionssicher archivierte oder besonders sensible Dokumente) - Konsistenz hat hier Vorrang vor Geschwindigkeit.
 - **Synchron auf Primärziel, asynchron auf Sekundärziele:** schnellere Antwortzeit, Nachreplikation mit Nachverfolgung offener Kopien (Retry-Queue, Alarmierung bei dauerhaftem Fehlschlag eines Ziels). **Werkseinstellung für den allgemeinen Arbeitsbetrieb**, wo Antwortzeit/Durchsatz Priorität hat und eine kurze Zeitspanne bis zur vollständigen Redundanz akzeptabel ist.
 - Beide Defaults sind reine Werkseinstellungen und pro Objekttyp/Ordner überschreibbar - die Entscheidung liegt letztlich beim Betreiber.
- Prüfsummen (z. B. SHA-256) werden pro Kopie erfasst und mit dem in der Shared DB hinterlegten Referenzwert abgeglichen - Basis für automatisierte Integritätsprüfungen (regelmäßiger Fixity-Check über alle Kopien).
- Lesezugriff: konfigurierbare Zielpriorität (z. B. „nächstgelegenes/schnellstes Ziel zuerst“, automatischer Fallback auf nächste Kopie bei Nichterreichbarkeit).
- Entfernen/Hinzufügen eines Backends aus einem bestehenden Ziel-Set ist ein eigener, auditierbarer Vorgang (Rebalancing: fehlende Kopien beim neuen Ziel nachziehen, bevor ein altes Ziel entfernt wird) - technisch verwandt mit dem Migrationsprozess aus 7.2.

Datenträger-Wechsel-Sensibilität

Ein besonders leicht zu übersehendes Risiko bei physischen/mountbaren Backends (NFS, lokales Filesystem, ggf. austauschbare Datenträger dahinter): ein Datenträger wird versehentlich getauscht, zurückgesetzt oder gegen ein leeres/falsches Ziel ausgetauscht, ohne dass der

Storage Service dies ohne Weiteres bemerken würde - ein stiller Rückfall auf ein falsches Medium wäre gerade für Redundanz/Integrität fatal.

- Jede vom Storage Service verwaltete Backend-Instanz legt bei erstmaliger Nutzung eine **Index-/Identitätsdatei** an ihrem Wurzelpunkt ab, die eine stabile, generierte **Geräte-ID** enthält.
- Beim Start prüft der Storage Service für jedes konfigurierte Backend, ob dessen Identitätsdatei vorhanden ist und mit der zuletzt bekannten Geräte-ID übereinstimmt.
- **Werkseinstellung: Startverweigerung**, sobald die Identitätsdatei eines konfigurierten Backends fehlt oder nicht übereinstimmt - der Storage Service kommt in diesem Fall bewusst nicht hoch, statt stillschweigend mit einem falschen/leeren Medium weiterzuarbeiten.
- **Admin-konfigurierbare Ausnahme:** Im Admin-UI (8) lässt sich hinterlegen, dass ein Start auch bei einem fehlenden/abweichenden Backend zulässig ist, **sofern mindestens ein anderes konfiguriertes Backend im Ziel-Set nachweislich unverändert** (Identitätsdatei passt) ist - der Service startet dann in einem bewusst degradierten Redundanzzustand statt komplett zu verweigern.
- In diesem degradierten Fall ist eine **Hintergrund-Replikation zwingend**, nicht optional: Der Storage Service zieht die fehlenden Kopien auf das neue/zurückgesetzte Ziel automatisch nach (dieselbe Nachreplikations-Infrastruktur wie beim regulären Rebalancing oben), bis die konfigurierte Redundanz (Quorum-/Ziel-Set-Vorgabe) wiederhergestellt ist - erst danach gilt das Ziel-Set wieder als vollständig gesund.
- Dieser gesamte Vorgang (Startverweigerung, Admin-Override, degradiert Start, Nachreplikationsfortschritt) ist wie jede andere Storage-Aktion auditierbar (3.4/5.3) und im Admin-UI als Status sichtbar.

3.7 Übersicht Kern-Services (Auszug)

Service	Verantwortung
Registry Service	Service Discovery, Lizenzvermittlung, Sensor-Registry für Monitoring
Plugin Orchestration Service	Verteilung/Skalierung von Plugins über die Serverlandschaft, zeitprofil-bewusste Platzierung
License Service	Lizenzverwaltung/-prüfung

Service	Verantwortung
Auth Service	Authentifizierung (lokal + AD), Session/Token
Permission Service	RBAC-Verwaltung, Rechteauswertung inkl. Vererbung, Bereichssperren, Not-Shutdown-Zustand
Document Service	Dokument-CRUD, Versionierung, Locking
Storage Service	Abstraktion über Speicherbackends (S3, NFS, ...), redundante Ablage, Fixity-Checks
Folder Service	Ordnerstruktur, Vererbungslogik
Object-Type Service	Objektdefinitionen, Attribute
Rule/Constraint Engine	Bedingungsprüfung bei Erstellung/Änderung
Workflow Engine	BPMN-2.0-Modellierung/-Import, Prozessausführung, SLA-Zeitüberwachung je Schritt
Case Service	Umlaufmappen-Verwaltung, Vorgangsreferenzen auf Dokumente, Archivierung von Umlaufmappen
Audit Service	Unveränderliches Ereignisprotokoll
Search Service	Volltextindex, Facettensuche
Rendering/Preview Service	Thumbnails, PDF/A-Konvertierung, Wasserzeichen, Ersatzdarstellungen (Barrierefreiheit/Ausfallsicherheit)
Virus-Scan Service	Upload-Prüfung
OCR Service	Texterkennung für gescannte/bildbasierte Dokumente, speist Suche und Ersatzdarstellungen
Signature Service	eIDAS-konforme elektronische Signatur (SES/AES/QES), Broker vor externen QTSPs
Notification Service	E-Mail/In-App/Webhook
Migration/Transfer	Ex-/Import einzelner Dokumente und ganzer Strukturen

Service	Verantwortung
Service	
Config Import/ Export Service	JSON-Konfigurationsverwaltung
Delta/Vergleichs- Service	Installationsübergreifender Konfigurationsvergleich (Basisinstanz vs. Vergleichsinstanz), konfigurierbarer Umfang, Ignore-Regex für Objektnamen
Query & Trace Service	Zentrale Abfrage-/Manipulationssprache über Audit-/Reporting-/Monitoring-Read-Modelle
Backup/Restore Orchestrator	Koordinierte Sicherung/Wiederherstellung über Storage/DB/ Konfiguration hinweg, Löschatgleich nach Restore, Wartungsmodus-Kopplung
Connector Services (CMIS, WebDAV, Custom, ...)	Anbindung externer Repositories
Federation Hub Service	Installationsübergreifende Workflow-Vermittlung (Adressbuch + Schaltzentrale, außerhalb einzelner Installationen)

3.8 Plugin-Orchestrierung & automatische Lastverteilung

Das "Dazustellen"-Prinzip (1., 3.2, 3.3) wird um eine aktive Orchestrierungsebene erweitert: Neue Services/Connectoren/Verarbeitungsschritte werden nicht nur passiv erkannt (Registry), sondern von einem eigenen **Plugin Orchestration Service** aktiv über die verfügbare Serverlandschaft verteilt und bei Bedarf automatisch skaliert - vergleichbar einem schlanken, fachlich angepassten Scheduler/Orchestrator (im Geiste von Kubernetes, aber mit DMS-spezifischer Platzierungslogik oberhalb bzw. in Kombination mit der eigentlichen Container-Plattform).

Grundprinzip

- Jedes "dazustellbare" Element (Connector, Rendering-Backend, Regel-Plugin, Storage-Backend-Adapter, Migrations-Worker, ...) wird als **Plugin-Paket** mit einem Manifest

beschrieben: benötigte Ressourcen (CPU/RAM/IO-Profil), unterstützte Skalierungsart (zustandslos horizontal skalierbar vs. genau eine Instanz), Abhängigkeiten zu anderen Services, und optional ein **Lastprofil** (siehe unten).

- Der Plugin Orchestration Service kennt über die Registry (3.2) den aktuellen Zustand aller Server/Knoten im Cluster (Auslastung, freie Kapazität, aktuell laufende Plugin-Instanzen) und entscheidet, auf welchem Knoten eine neue oder zusätzliche Instanz gestartet wird.
- Administrierende stellen ein Plugin lediglich bereit (Upload/Registrierung des Pakets) - die Platzierung, Skalierung und ggf. Migration auf einen anderen Knoten bei Lastspitzen erfolgt automatisch, ohne manuelle Zuweisung "Plugin X läuft auf Server Y".

Zeitprofil-bewusste Platzierung (Kernanforderung)

- Viele Arbeiten in einem DMS sind nicht gleichmäßig über den Tag verteilt (z. B. nächtliche Fixity-Checks im Storage Service, Reporting-Aggregationen, Massen-Migrationsjobs, AD-Synchronisation außerhalb der Kernarbeitszeit, Retentions-/Löschläufe).
- Plugins können ein **Lastprofil** hinterlegen (z. B. "läuft bevorzugt nachts", "läuft periodisch alle X Minuten", "läuft interaktiv/on-demand mit Nutzerzugriff tagsüber") - der Orchestrator nutzt dieses Profil, um sich zeitlich ergänzende Jobs bevorzugt auf denselben Knoten zu legen (z. B. ein tagsüber stark ausgelasteter interaktiver Service und ein nachts laufender Batch-Job teilen sich denselben Server, statt dass für beide dauerhaft getrennte Kapazität vorgehalten wird).
- Auch ohne explizit hinterlegtes Lastprofil beobachtet der Orchestrator die tatsächliche historische Auslastung je Plugin-Typ (einfache Zeitreihenauswertung auf Basis der Monitoring-Sensoren, siehe 10.1) und leitet daraus mit der Zeit ein implizites Profil ab - die Konfiguration ist also optional, nicht zwingend.
- Umverteilung bestehender Instanzen (Rebalancing) erfolgt kontrolliert und schonend: laufende Aufträge werden nicht hart abgebrochen, sondern neue Zuteilungen greifen für neue Instanzen/Jobs; laufende Prozesse dürfen regulär abschließen (Drain-Verhalten), bevor eine Instanz verschoben/beendet wird.

Platzierungsalgorithmus: Plattform-Scheduler bevorzugt, First Fit Decreasing als Fallback

- Läuft die Installation auf einer Plattform mit eigenem, geeignetem Scheduler (z. B. der Kubernetes-/OpenShift-Scheduler), **vertraut der Plugin Orchestration Service dessen Platzierungsentscheidungen** und beschränkt sich auf die DMS-spezifische Zusatzlogik (Zeitprofil-Zuordnung, Lizenzprüfung vor Platzierung, Kommunikation mit der Registry) - keine parallele, konkurrierende Platzierungsentscheidung zum ohnehin vorhandenen,

ausgereiften Plattform-Scheduler.

- Steht kein solcher Plattform-Scheduler zur Verfügung (z. B. einfacher Docker-/VM-Betrieb ohne Orchestrierungsschicht), übernimmt der Plugin Orchestration Service die Platzierung selbst, als **Fallback nach dem First-Fit-Decreasing-Verfahren (FFD)**: Neue/umzuverteilende Plugin-Instanzen werden absteigend nach ihrem Ressourcenbedarf sortiert und jede Instanz wird dem ersten Knoten zugewiesen, auf dem sie noch ausreichend freie Kapazität vorfindet - ein bewährter, einfach nachvollziehbarer Bin-Packing-Ansatz mit guter Praxistauglichkeit, ohne die Komplexität eines vollständigen eigenen Schedulers nachzubauen. Die Zeitprofil-bewusste Gruppierung (s. o.) fließt dabei als zusätzliches Sortier-/Gruppierungskriterium vor der reinen Ressourcengrößen-Sortierung ein.

Herkunft der Ressourcen-/Auslastungsdaten

- Wo die zugrunde liegende Plattform selbst bereits verlässliche Ressourcen-/Auslastungsdaten bereitstellt (z. B. der Metrics-Server/Scheduler einer Kubernetes-/OpenShift-Umgebung), **vertraut der Plugin Orchestration Service bevorzugt darauf**, statt Daten redundant selbst zu erheben - vermeidet doppelten Overhead und nutzt bewährte, plattformnative Mechanismen.
- Nur wenn die Plattform diese Daten nicht oder nicht in ausreichender Granularität liefert, erhebt der Orchestrator sie über einen eigenen Mechanismus (im Kern dieselbe Sensor-Infrastruktur wie in 10.1 beschrieben).
- **Cold-Start-Platzierung** (neues Plugin ohne jede historische Auslastung): Der Orchestrator greift zunächst auf **statische Richtwerte zurück, die das Plugin selbst im Manifest bereitstellen kann** (z. B. erwarteter CPU-/RAM-Bedarf je Instanz). Stellt ein Plugin keine solchen Werte bereit, verwendet der Orchestrator ersatzweise den **Median der tatsächlich beobachteten Ressourcennutzung aller aktuell registrierten Plugins desselben Typs/derselben Kategorie** als vorläufige Schätzung, bis genug eigene Beobachtungsdaten für das neue Plugin vorliegen und die Schätzung durch echte Messwerte abgelöst wird.

Abgrenzung zur Registry

- Registry (3.2) beantwortet die Frage "wer ist gerade erreichbar und lizenziert" (Control Flow, reaktiv).
- Plugin Orchestration Service beantwortet die Frage "wo sollte etwas laufen, damit die Serverlandschaft insgesamt optimal ausgelastet ist" (aktive Steuerung). Beide Services

arbeiten eng zusammen: der Orchestrator meldet Platzierungsentscheidungen an die Registry, damit andere Services die aktuell erreichbaren Instanzen weiterhin korrekt auflösen können.

- Auch der Plugin Orchestration Service selbst ist - konsequent zu Ende gedacht - ein Service, dessen Entscheidungen auditiert werden (welches Plugin wurde wann auf welchem Knoten platziert/verschoben) und dessen Aktivität über die Sensor-/Monitoring-Schicht (10.1) beobachtbar ist.

3.9 OCR-Service (Texterkennung)

Fester Bestandteil der Eingangserfassung, nicht nachträglich optional: Gescannte oder bildbasierte Dokumente (z. B. eingescannte Papierpost, Faxe, Bild-PDFs ohne Textlayer) durchlaufen automatisiert eine Texterkennung, damit sie durchsuchbar werden und inhaltlich weiterverarbeitet werden können (z. B. für die Constraint Engine, 2.2, oder die Query-Konsole, 6.1).

- **Pipeline-Einordnung:** Upload → Virensan → **OCR (falls Dokument keinen nutzbaren Textlayer enthält)** → Rendering/Preview (3.7) → Such-Indexierung (Search Service). Ob ein Dokument OCR benötigt, wird automatisch anhand des vorhandenen Textlayers erkannt, nicht pauschal für jeden Dateityp erzwungen.
- **Plugin-basiert** (3.3/3.8): Die konkrete OCR-Engine ist austauschbar, analog zu Storage-Backends und Connectoren - neue/bessere Engines lassen sich später ergänzen, ohne den Kern zu ändern.
- **Konkrete Empfehlung** (siehe 1a): **PaddleOCR** als Standard-Engine (Apache-2.0-lizenziert, passt zum Open-Source-Anspruch) - deutlich stärker als klassisches Tesseract bei komplexen Layouts, Tabellen und mehrsprachigen Dokumenten, was für Verwaltungs-/Geschäftsdokumente (Formulare, Tabellen, gemischte Layouts) relevant ist. **Tesseract** bleibt als ressourcenschonende Alternative für einfache Fälle oder besonders eingeschränkte Umgebungen (geringerer Ressourcenbedarf, keine GPU nötig) über dasselbe Plugin-Interface wählbar.
- Das Ergebnis (OCR-Textlayer) wird **mit der jeweiligen Dokumentversion verknüpft und dauerhaft gespeichert** (nicht bei jeder Suche neu berechnet) und speist direkt den Search Service (3.7).
- Der erzeugte durchsuchbare Textlayer kann zugleich als Basis für eine text-basierte Ersatzdarstellung (2.4) dienen (z. B. reine .txt-Extraktion eines gescannten Dokuments) - dieselbe Infrastruktur wird für zwei Zwecke genutzt: Suche und Barrierefreiheit/

Ausfallsicherheit.

- **Konfigurierbarkeit (kein Immer-an-Zwang):** Obwohl OCR laut obigem Absatz fester Bestandteil der Eingangserfassung ist, bleibt der Betrieb der Komponente selbst konfigurierbar, statt sie in jeder Installation zwingend vorzuhalten:
 - **Vollständig deaktivierbar** (`ocrEnabled: false`): Ist OCR für eine Installation nicht gewünscht/nicht lizenziert/nicht nötig (z. B. weil ausschließlich bereits textbasierte Dokumente eingehen), muss der OCR-Service selbst **nicht deployt werden** - konsistent mit dem "Dazustellen"-Prinzip (1.): Abwesenheit eines Plugins ist ein regulärer Zustand, kein Fehlerfall. Ohne aktiven OCR-Service bleiben entsprechende Dokumente ohne Textlayer (Suche/Ersatzdarstellung greifen dann nur auf ggf. vorhandenen nativen Text zurück).
 - **Maximale Wortobergrenze:** konfigurierbarer Schwellwert, ab dem ein Dokument (geschätzt anhand Seiten-/Bildgröße) als "zu groß für OCR" übersprungen wird, statt unbegrenzt Rechenzeit in sehr große Scans zu investieren - Kosten-/Performance-Schutzventil, kein fachliches Muss.
 - **Verarbeitungs-Batch-Size:** konfigurierbar, wie viele Seiten/Dokumente je Engine-Aufruf/Worker-Zyklus gemeinsam verarbeitet werden - Stellschraube für Durchsatz vs. Ressourcenverbrauch (Speicherbedarf pro Batch), je Installationsgröße unterschiedlich sinnvoll.
 - Alle drei Einstellungen sind Admin-UI-Konfiguration (8), keine Code-/Deployment-Änderung.
- OCR-Ergebnisse werden wie jede andere Dokumentverarbeitung auditert (5.3); bei besonders fehleranfälligen/unsicheren Erkennungen (niedriger Konfidenzwert der Engine) kann optional eine manuelle Nachprüfung als BPMN-Prozessschritt (7.1) angestoßen werden, statt fehlerhafte OCR-Ergebnisse unkommentiert durchzureichen.

3.10 Signatur-Anbindung (eIDAS-konforme elektronische Signatur)

Elektronische Signatur wird als fester Bestandteil von Freigabeprozessen unterstützt - technisch als **Broker vor externen, zertifizierten Vertrauensdiensteanbietern (Qualified Trust Service Providern, QTSP)**, nicht als eigener Vertrauensdienst: Die rechtliche Anerkennung einer qualifizierten elektronischen Signatur setzt eine eigene Akkreditierung nach eIDAS voraus, die kein Softwareprodukt selbst "mitbringen" kann - das System integriert stattdessen bereits akkreditierte Anbieter, ähnlich wie bei den übrigen Connector-Architekturen (3.3).

- **Signature Service** als eigener Kern-Service, der über austauschbare **Signature-Provider-Connectoren** (Plugin-Prinzip wie bei CMIS/WebDAV, 3.3) mit externen QTSPs kommuniziert (z. B. für qualifizierte Signaturen) oder - für niedrigere Vertrauensniveaus - auch rein intern mit systemeigenen Schlüsseln arbeiten kann.
 - **Unterstützte Signaturniveaus nach eIDAS**, konfigurierbar je Objekttyp/Prozessschritt, welches Mindestniveau erforderlich ist:
 - **SES (Simple Electronic Signature)**: einfache elektronische Bestätigung, systemintern erzeugbar, kein externer Anbieter nötig.
 - **AES (Advanced Electronic Signature)**: eindeutig einer Person zuordenbar, üblicherweise über einen externen Anbieter oder ein Zertifikat.
 - **QES (Qualified Electronic Signature)**: höchstes Niveau, zwingend über einen akkreditierten QTSP, rechtlich der handschriftlichen Unterschrift gleichgestellt.
 - **Format**: PAdES für PDF-Dokumente (Standardfall), XAdES/CAAdES für andere Formate, je nach Dokumenttyp automatisch gewählt.
 - **Einbindung in die BPMN-Prozesse (7.1)**: Ein neuer Task-Typ "**Signature Task**" ergänzt Manual/Automatic Tasks - ein Prozessschritt kann eine Signatur einer berechtigten Person verlangen, bevor der Prozess weiterläuft; die Signatur wird an die konkrete Dokumentversion gebunden (2.1a).
 - **Verifikation**: Signaturen sind jederzeit auf Gültigkeit prüfbar (Zertifikatskette, Sperrstatus, Zeitstempel) - wichtig gerade für Langzeitarchivierung/Aussonderung (5.6), wo die Gültigkeit einer Signatur auch Jahre später noch nachweisbar bleiben muss (PAdES-B-LTA-Profil mit fortlaufender Zeitstempelkette deckt genau diesen Fall ab).
 - **Konkrete Bibliotheksempfehlung** (siehe 1a): **pyHanko** für die PDF-seitige Erzeugung/ Einbettung/Prüfung von PAdES-Signaturen (unterstützt alle gängigen PAdES-Baseline-Profile B-B/B-T/B-LT/B-LTA sowie PKCS#11 für Hardware-Token/HSM-Anbindung) - übernimmt die technische PDF-Signaturarbeit, während die eigentliche Vertrauensdienstleistung (Zertifikatsausstellung, qualifizierte Signaturerstellung) beim angebundenen externen QTSP verbleibt.
 - Signierte Dokumente/deren Signaturstatus sind Teil der Revisionssicherheit (5.1): eine nachträgliche Veränderung des signierten Inhalts wird durch die kryptografische Signatur selbst erkennbar, zusätzlich zum ohnehin bestehenden Audit-Trail (5.3).
-

3a. Mehrfachinstallationen (Installation ≠ Mandant)

Entscheidung: **keine Multi-Tenant-Architektur innerhalb einer Installation**, sondern ein Kunde kann **mehrere vollständig unabhängige Installationen** desselben Systems betreiben (z. B. ein IT-Dienstleister, der je einen eigenen Systemstand für zwei seiner Endkunden bereitstellt).

- Jede Installation ist vollständig isoliert: eigene Shared-DB-Instanz (oder zumindest eigener Satz an Schemata), eigener Event-Bus, eigene Registry-Instanz, eigener Storage-Zielsatz. Kein gemeinsamer Datenzugriff zwischen Installationen.
- Das vermeidet die Komplexität und die Risiken von Mandantentrennung *innerhalb* einer Instanz (Datenlecks zwischen Mandanten, komplexere Rechteprüfung) und passt zum "Dazustellen"-Prinzip: eine neue Installation ist im Kern ein weiteres komplettes Set an Services, das hochgefahren wird.
- **Zentrale Verwaltungsebene (optional, separater Baustein):** Ein übergeordneter **Fleet-/Lizenz-Management-Service** (unabhängig von den einzelnen Installationen) ermöglicht es einem Betreiber, mehrere Installationen zu überblicken - insbesondere Lizenzvergabe/-verlängerung (siehe 9.2), grundlegende Health-Übersicht **und zentrales Provisioning neuer Installationen** (Bereitstellung einer neuen Installation aus einer Konfigurationsvorlage heraus, angestoßen über diese zentrale Ebene statt manuell je Installation). Dieser Service hat aber **keinen** Zugriff auf Dokumenteninhalte einzelner Installationen, nur auf deren Lizenz-/Statusmeldungen.
- **Flexible Deployment-Topologie für den License Service:** Beide folgenden Betriebsmodi werden unterstützt, ohne dass sich am Kern des License Service etwas ändert:
 - **Vollständig isoliert/standalone:** Eine Installation betreibt ihren eigenen License Service z. B. in einem eigenen, abgeschotteten Namespace einer Container-Plattform (etwa OpenShift/OCP) - keine Abhängigkeit von einer externen Instanz, maximale Isolation.
 - **Zentraler License Service für mehrere Installationen:** Ein License Service läuft in einem zentralen Namespace/Cluster-Bereich, und mehrere Installationen (in eigenen Namespaces) rufen diesen zentralen Dienst für ihre Lizenzprüfung auf, statt jeweils einen eigenen zu betreiben - passend zum Fleet-Management-Szenario, reduziert den Betriebsaufwand bei vielen Installationen.
 - Welches Modell greift, ist eine reine Konfigurations-/Deploymententscheidung der Registry (3.2) bei der Registrierung des License Service - der Rest des Systems merkt

technisch keinen Unterschied.

- Konfigurationsexport/-import (7.3) ist der vorgesehene Weg, um eine neue Installation aus dem Konfigurationsstand einer bestehenden aufzusetzen (Cloning), nicht ein geteilter Datenbestand.
 - Jede Installation registriert sich beim License Service (ggf. beim übergeordneten Fleet-Management) mit einer eigenen Installations-ID, gegen die ihre Lizenz geprüft wird.
-

4. Berechtigungsmodell

4.1 RBAC als Grundlage

- Rollen werden zentral im Permission Service definiert und können objekttyp- sowie ordnerbezogen zugewiesen werden.
- **Vererbung:** Rechte vererben sich von übergeordneten Ordnern nach unten, mit expliziter Override-Möglichkeit auf tieferen Ebenen (Standard-DMS-Verhalten, z. B. wie bei SharePoint/Alfresco).
- Für Performance bei tiefen Hierarchien: effektive Rechte werden nicht bei jeder Anfrage neu über die gesamte Kette berechnet, sondern in einem materialisierten, ereignisgetrieben aktualisierten Cache gehalten (Invalidierung über Events aus 3.4, wenn sich Rechte oder Struktur ändern).
- Rollenzuweisung sowohl an einzelne Nutzer als auch an Gruppen (lokal oder AD-Gruppen, siehe 4.4).

4.2 Dokumentensperre bei Bearbeitung (Locking)

- Öffnet ein Nutzer ein Dokument aus einer Anwendung heraus (z. B. Word über WebDAV/CMIS-Anbindung) zur Bearbeitung, wird automatisch ein **Bearbeitungs-Lock** gesetzt, der andere Schreibzugriffe verhindert (Lesezugriff bleibt je nach Konfiguration möglich).
- Der Lock wird vom Document Service verwaltet und ist an Nutzer + Session gebunden.
- **Freigabe-Mechanismen:**
 - regulär durch den Nutzer bei Abschluss der Bearbeitung (Check-in),
 - automatisch nach konfigurierbarem Timeout ohne Aktivität (z. B. Session/Verbindung abgebrochen),

- **administrativer Force-Unlock:** Rolle mit entsprechender Berechtigung kann eine Sperre manuell aufheben (z. B. bei länger abwesenden Mitarbeitenden). Dieser Vorgang ist selbst ein besonders sensibler Audit-Fall (siehe 4.3) und sollte optional dem 4-Augen-Prinzip unterliegen.

Konfliktbehandlung bei Force-Unlock (konkretisiert):

Ziel: Der ursprüngliche Bearbeiter darf nie stillschweigend Arbeit verlieren, auch wenn die Sperre administrativ aufgehoben wurde.

1. Beim Force-Unlock wird der Lock nicht sofort vollständig entfernt, sondern in einen Zustand **"aufgehoben, aber überwacht"** versetzt: Das Dokument ist für andere Nutzer wieder bearbeitbar, der ursprüngliche Lock-Inhaber wird aber per Notification informiert, dass seine Sperre administrativ aufgehoben wurde (inkl. Grund, sofern erfasst).
2. Lädt in der Zwischenzeit ein anderer Nutzer eine neue Version hoch (reguläres Check-in), wird diese als neue Hauptversion übernommen - der Vorgang ist Teil der normalen Versionierung.
3. Versucht der **ursprüngliche** Bearbeiter danach ebenfalls einen Stand hochzuladen (z. B. weil er von der Aufhebung nichts wusste oder sein Client die Datei noch offen hatte), erkennt der Document Service anhand der Versionskette, dass sein Ausgangsstand nicht mehr die aktuelle Version ist. Sein Upload wird **nicht überschreibend** eingespielt, sondern als **Konflikt-Kopie** (z. B. „Dokument_conflict__“) neben der aktuellen Version abgelegt.
4. Beide Stände (aktuelle Hauptversion + Konfliktkopie) werden im Audit-Trail und in der UI sichtbar verknüpft; eine manuelle Zusammenführung durch die Beteiligten oder eine berechtigte Person ist über die reguläre Versionsverwaltung möglich.
5. Der gesamte Vorgang (Force-Unlock, Benachrichtigung, ggf. entstandene Konfliktkopie) ist vollständig auditiert und kann - wie in 4.3 beschrieben - optional dem Vier-Augen-Prinzip unterliegen.

Damit geht bei einem Force-Unlock nie eine Bearbeitung ohne Sichtbarkeit verloren; im schlimmsten Fall entsteht eine sichtbare Konfliktkopie statt eines stillen Datenverlusts.

4.3 4-Augen-Prinzip (konfigurierbar)

- Konfigurierbar pro Aktionstyp, nicht global erzwungen. Beispiele für Aktionstypen, die optional ein Vier-Augen-Prinzip erfordern können:
 - Freigabe eines Dokuments/Workflow-Schritts
 - Löschung (endgültig, nach Aufbewahrungsfrist)

- Änderung von Berechtigungen/Rollen
- Force-Unlock eines Dokuments
- Änderung von Objekttyp-Definitionen oder Constraints
- Import/Export von Konfiguration
- Start eines Migrations-/Transfer-Vorgangs
- Technisch als generischer **Approval-Mechanismus** im Permission Service/Workflow Engine gelöst: eine Aktion mit aktivem Vier-Augen-Flag erzeugt einen Freigabe-Request, der von einer zweiten berechtigten Person (nicht identisch mit Initiator) bestätigt werden muss, bevor sie wirksam wird.
- Konfiguration erfolgt pro Mandant/Instanz und ist Teil des JSON-Konfigurationsexports (siehe 7).

4.4 Nutzerverwaltung: AD + Lokal

- Auth Service unterstützt parallel:
 - **LDAP/Active-Directory-Anbindung** (Nutzer- und Gruppensynchronisation, Login via AD-Credentials/Kerberos/SSO optional),
 - **lokale Konten** (für externe Nutzer, Dienstkonten, Systeme ohne AD-Anbindung).
- Gruppenmitgliedschaften aus AD werden auf interne Rollen gemappt. Dieses Mapping ist **vollständig konfigurierbar** und nicht fest im Code hinterlegt:
 - Administrierbar über die Admin-UI und Teil des JSON-Konfigurationsexports (7.3), damit ein Mapping zwischen Installationen übertragen werden kann.
 - Unterstützt einfache 1:1-Zuordnung (AD-Gruppe → interne Rolle) ebenso wie komplexere Regeln (z. B. AD-Gruppe X **und** Attribut Y → Rolle Z; mehrere AD-Gruppen → eine gemeinsame Rolle).
 - Änderungen am Mapping wirken sich erst nach expliziter Freigabe/Speicherung aus (kein Live-Editing mit sofortiger Breitenwirkung ohne Kontrolle) und sind selbst auditiert.
 - Bei nicht gemapptem AD-Gruppen: konfigurierbares Default-Verhalten (keine Rolle vergeben vs. definierte Standardrolle).
- Synchronisationsintervall konfigurierbar; Deaktivierung/Löschung von AD-Nutzern muss sich zeitnah auf Zugriffsrechte auswirken.

Konkrete Technologiewahl: OpenID Connect (OIDC) auf Basis von Keycloak

- Als Protokoll wird durchgängig **OpenID Connect** (auf OAuth2 aufbauend) verwendet: Der

Auth Service erhält/validiert JWT-Access-Tokens, die zustandslose Rechteprüfung durch jeden einzelnen Microservice ermöglichen (passend zum Leitprinzip der konsequenten Zustandslosigkeit, 1.) statt eine zentrale Session-Abfrage bei jedem Aufruf zu erfordern.

- Als konkreter Identity-Provider dahinter wird **Keycloak** eingesetzt statt AD-/LDAP-Föderation, Multi-Faktor-Authentifizierung und Session-Verwaltung selbst zu entwickeln: Keycloak deckt OIDC, SAML 2.0, LDAP-/AD-Federation und Identity Brokering bereits vollständig ab, ist unter Apache-2.0 lizenziert (passt zum Open-Source-Anspruch) und nutzt für den Produktivbetrieb ebenfalls PostgreSQL (3.1) - kein zusätzliches Datenbanksystem nötig.
- Der Auth Service des Konzepts wird dadurch zu einem **schlanken Adapter/Broker vor Keycloak**, der die Ergebnisse (Token, Gruppenzugehörigkeiten) ins interne RBAC-Modell (4.1) übersetzt, statt eine eigene IAM-Logik zu implementieren - reduziert Entwicklungsaufwand und Sicherheitsrisiko gegenüber einer Eigenentwicklung erheblich.
- **SAML 2.0** steht darüber für Installationen mit älteren ADFS-/SAML-basierten Föderationsanforderungen zur Verfügung, ebenfalls über Keycloak, ohne eigene Zusatzentwicklung.

4.5 Custom Bedingungen (Constraint Engine)

- Regel-Engine wertet bei Erstellung/Änderung von Dokumenten/Ordern die im Objekttyp hinterlegten Bedingungen aus (siehe Beispiel in 2.2).
- Unterstützt mindestens: Pflichtfelder, bedingte Pflichtfelder (abhängig von anderen Attributwerten), Musterprüfung für Namen/Werte (Regex), Wertebereiche, Verweise auf andere Objekte.
- Sollte erweiterbar sein um eigene Validierungs-Plugins (ähnlich der Connector-Idee: neue Regeltypen als "dazustellbare" Erweiterung).

4.6 Superuser-Prinzip (Break-Glass) & domänengetrennte Admin-Rollen

Ziel: Die Angriffsfläche eines kompromittierten administrativen Kontos minimieren, indem es im Normalbetrieb gar keinen dauerhaft nutzbaren Vollzugriffs-Account gibt.

Superuser als deaktivierter Break-Glass-Zugang

- Nach der Ersteinrichtung wird der Superuser-Zugang (Zugriff auf wirklich alle Funktionen

ohne Einschränkung, inkl. Manipulationsmodus der Query-Konsole aus 6.1) standardmäßig **deaktiviert**.

- Reaktivierung erfordert **zwingend das Vier-Augen-Prinzip**, wobei die Berechtigung zur Freigabe nicht an zwei fest benannte Einzelkonten gebunden ist, sondern über **Gruppenzugehörigkeit** definiert wird: Zwei unterschiedliche Mitglieder einer (oder mehrerer) dafür vorgesehenen Berechtigungsgruppe(n) müssen die Aktivierung gemeinsam bestätigen. Die Gruppe sollte im Idealfall mehr als zwei Mitglieder umfassen, damit die Vier-Augen-Regel auch bei Abwesenheit Einzelner hart durchsetzbar bleibt, statt bei Ausfall einer der beiden fest benannten Personen zum Single Point of Failure zu werden.
- Die Aktivierung ist zeitlich befristet und deaktiviert sich **zusätzlich automatisch bei Inaktivität** (konfigurierbares Zeitfenster, **Standardwert: 10 Minuten** ohne Aktivität als Superuser) - unabhängig von einem eventuell separat konfigurierten maximalen Zeitfenster der Aktivierung insgesamt. Damit bleibt ein vergessenes, aber ungenutztes Superuser-Fenster nicht unnötig lange offen.
- Jede Aktivierung, jeder Login und jede im aktivierten Zustand durchgeführte Aktion wird mit erhöhter Priorität auditert und kann optional zusätzlich eine sofortige Benachrichtigung an eine definierte Sicherheitsverantwortliche/einen Sicherheitsverantwortlichen auslösen.

Abgespeckte, domänengetrennte Admin-Rollen für den Alltag

- Für den laufenden Administrationsbetrieb existieren stattdessen von Anfang an mehrere **vordefinierte, im Funktionsumfang eingeschränkte Admin-Rollen**, die jeweils nur eine fachliche Domäne abdecken - z. B. getrennte Rollen für Nutzer-/Rechteverwaltung, für Objekttyp-/Workflow-Konfiguration, für Storage-/Backend-Verwaltung, für Lizenzverwaltung, für die Query-Konsole (6.1, i. d. R. nur lesend), sowie **Löschadministration** und, davon personell trennbar, **Löschadministration (Verschlusssachen)** für die endgültige Löschung aus den beiden getrennten Papierkorb-Bereichen (2.5).
- Diese Rollen sind **standardmäßig mitgeliefert** (keine Pflicht, sie erst selbst zu modellieren) und lassen sich wie jede andere RBAC-Rolle weiter anpassen/einschränken.
- Wird ein einzelnes dieser abgespeckten Admin-Konten kompromittiert, ist der mögliche Schaden strukturell auf dessen Domäne begrenzt - insbesondere kein Zugriff auf die Query-Konsolen-Manipulationsfunktion (6.1) oder auf domänenfremde Bereiche.
- Zusammenspiel mit 4.3 (Vier-Augen-Prinzip): einzelne Aktionen auch dieser abgespeckten

Rollen können weiterhin zusätzlich dem Vier-Augen-Prinzip unterliegen, wo das konfiguriert ist - die Domänentrennung und das Vier-Augen-Prinzip sind unabhängige, sich ergänzende Schutzmechanismen.

4.7 Bereichssperren (Aussperren regulärer Nutzer aus Ordnerbereichen)

Ergänzend zum Locking einzelner Dokumente (4.2) kann ein **ganzer Ordnerbereich** (inkl. aller Unterordner) gezielt für reguläre Nutzer gesperrt werden, sodass dort - unabhängig von den sonst geltenden RBAC-Rechten - vorübergehend keine Schreib- (konfigurierbar auch: keine Lese-)Zugriffe mehr möglich sind.

- Anwendungsfälle: bevorstehende Migration/Aussonderung eines Bereichs (7.2/5.6), Verdacht auf Datenintegritätsprobleme in einem Teilbaum, organisatorische Sperrungen (z. B. während einer Revision/Prüfung eines Aktenbereichs).
- Eine Bereichssperre ist eine eigene, klar von der regulären Rechtevergabe unterschiedene Konstruktion: sie **überlagert** RBAC-Rechte temporär, statt sie zu verändern - nach Aufhebung der Sperre gelten unmittelbar wieder die ursprünglichen Rechte, ohne dass diese vorher manuell entzogen und wieder vergeben werden mussten.
- Wer eine Bereichssperre setzen/aufheben darf, ist selbst RBAC-gesteuert (typischerweise eine der abgespeckten Admin-Rollen aus 4.6, ggf. mit Vier-Augen-Pflicht analog 4.3) und wird auditiert (5.3).
- Betroffene Nutzer erhalten beim Zugriffsversuch eine klare Rückmeldung samt - falls hinterlegt - Grund und voraussichtlicher Dauer der Sperre, statt eines unspezifischen Berechtigungsfehlers.

4.8 Not-Shutdown (systemweite Notfallsperre & Wartungsmodus)

Ein System weites, kurzfristig auslösbares Sperr-Werkzeug für den Fall eines Verdachts auf unautorisierten Zugriff - Ziel ist die schnellstmögliche Schadensbegrenzung, notfalls auf Kosten der Verfügbarkeit.

- **Auslösung:** Welche Rolle(n)/Gruppe(n) einen Not-Shutdown auslösen dürfen, ist **frei konfigurierbar** je Installation (keine fest im System verdrahtete Rolle) - typischerweise, aber nicht zwingend, eine kleine, eigens dafür vorgesehene Sicherheits-/Notfallrolle. Ob

die Auslösung selbst zusätzlich dem **Vier-Augen-Prinzip** unterliegt, ist ebenfalls **optional konfigurierbar** (analog 4.3): Manche Installationen werden hier bewusst auf eine zweite Freigabe verzichten wollen, damit im akuten Verdachtsfall keine Verzögerung entsteht; andere - z. B. um Missbrauch der Notfallfunktion selbst vorzubeugen - werden auch hier eine zweite Bestätigung verlangen. Die Auslösung erfolgt über die Admin-UI, das CLI-Tool (6.2) oder einen dedizierten Notfall-Endpunkt.

- **Wirkung, sofort und systemweit:**

- Alle regulären Nutzersitzungen werden beendet bzw. eingefroren; neue Logins außer für den Superuser werden abgelehnt.
- Alle laufenden Workflow-Instanzen, geplanten/periodischen Jobs (inkl. Migrations-/Aussonderungs-/AD-Sync-Läufe) und **alle Plugin-Instanzen** (3.8) werden angehalten - kontrolliert dort, wo ein sauberer Stopp ohne Datenverlust möglich ist, sonst hart, mit anschließender Konsistenzprüfung beim Wiederanlauf.
- Schreibender Zugriff auf Storage (3.6) und Shared DB wird für alle Services außer den für die Notfallreaktion selbst nötigen unterbunden.
- Föderierte Vorgänge über den Federation Hub (7.4) werden pausiert, nicht abgebrochen - offene Übergaben bleiben im Wartestand, statt inkonsistent abzureißen.

- **Wartungsmodus als Folgezustand:** Nach Auslösung befindet sich das System in einem expliziten Wartungszustand, der von jedem regulären UI/CLI-Zugriff klar signalisiert wird. Aus diesem Zustand heraus ist **ausschließlich der aktivierte Superuser** (4.6 - die Notfallsperre erzwingt/erlaubt hierfür die reguläre Break-Glass-Aktivierung) handlungsfähig, um die Lage zu prüfen und die Sperre gezielt wieder aufzuheben.
 - Der gesamte Vorgang (Auslösung, Zeitpunkt, auslösende Person/Gruppe, spätere Aufhebung) wird mit höchster Priorität auditiert (5.3) und kann automatisch eine Alarmierung an hinterlegte Sicherheitsverantwortliche auslösen.
 - Abgrenzung zur Bereichssperre (4.7): Der Not-Shutdown ist systemweit und für den akuten Sicherheitsvorfall gedacht, die Bereichssperre ist gezielt/organisatorisch und betrifft einen begrenzten Ordnerbereich im laufenden Regelbetrieb - beide Mechanismen sind unabhängig voneinander nutzbar.
-

5. Compliance & Auditierbarkeit

5.1 Revisionssicherheit

- Orientierung an **GoBD**-Grundsätzen: Nachvollziehbarkeit, Vollständigkeit, Richtigkeit, zeitgerechte Erfassung, Ordnung, Unveränderbarkeit.
- Als fachliche Referenzarchitektur bietet sich die BSI-Richtlinie **TR-ESOR** an (vertrauenswürdige Langzeitspeicherung), auch wenn keine 1:1-Umsetzung nötig ist.
- Einmal archivierte/freigegebene Dokumentversionen sind unveränderlich (WORM-artiges Verhalten auf Storage-Ebene, z. B. Objektspeicher mit Object Lock/Retention). **Einzige bewusste Ausnahme**: die verpflichtende, physische Zwangslöschung nach Frist (5.2a), wo eine gesetzliche Löschpflicht das Unveränderlichkeitsprinzip im Einzelfall gezielt und nachvollziehbar überschreibt.

5.2 Aufbewahrung, Löschung, Legal Hold

- Aufbewahrungsfristen sind pro Objekttyp/Ordner konfigurierbar.
- **Legal Hold**: Möglichkeit, ein Dokument/eine Struktur unabhängig von der regulären Frist zu sperren (z. B. bei laufendem Rechtsstreit) - überschreibt die reguläre Löschfrist, bis der Hold aufgehoben wird.
- **DSGVO-Spannungsfeld**: Löschpflicht bei personenbezogenen Daten vs. Aufbewahrungspflicht. Lösungsansatz: Pseudonymisierung/Schwärzung einzelner personenbezogener Attribute statt Hardlöschung des gesamten Dokuments, wo eine Aufbewahrungspflicht besteht; echte Löschung nur, wo keine gesetzliche Pflicht entgegensteht.
- Papierkorb/Soft-Delete mit konfigurierbarer Wiederherstellungsfrist vor endgültiger Löschung.

5.2a Verpflichtende Löschung nach Frist (Zwangslöschung)

Ergänzend zu Aufbewahrungsfristen (die primär regeln, wie *lange* etwas mindestens erhalten bleiben muss) unterstützt das System das umgekehrte, ebenso wichtige Szenario: Dokumente oder Umlaufmappen, die aus Datenschutz- oder anderen rechtlichen Gründen nach Ablauf einer Frist **tatsächlich gelöscht werden müssen** - vom System durchsetzbar, nicht nur als organisatorischer Hinweis.

Konfiguration je Dokument/Umlaufmappe

- **Löschdatum bzw. "Löschen nach Zeitraum"**: ein konkretes Datum oder eine relative Frist (z. B. "5 Jahre nach Erstellung", "1 Jahr nach Vorgangsabschluss") kann je Dokument oder Umlaufmappe hinterlegt werden - entweder manuell gesetzt oder automatisch aus dem Objekttyp (2.2) übernommen, wenn der Objekttyp eine Standardfrist definiert.
- **Checkbox "vollständig löschen" (physische Löschung)**: Standardverhalten ohne diese Markierung ist die reguläre, bereits bestehende Soft-Delete-/Archivierungslogik (5.2/5.6) - das Objekt verschwindet aus der aktiven Nutzung, bleibt aber im Sinne der Revisionssicherheit (5.1) grundsätzlich erhalten. Ist die Checkbox gesetzt, wird stattdessen eine **echte, physische Löschung** durchgeführt: Das Objekt (inkl. aller Versionen, 2.1a, und aller redundanten Kopien über sämtliche konfigurierten Storage-Backends, 3.6) wird unwiederbringlich entfernt.
 - Das ist die eine bewusste, eng kontrollierte **Ausnahme vom sonst geltenden Unveränderlichkeits-/Revisionssicherheits-Prinzip** (5.1) - gerechtfertigt dadurch, dass hier eine gesetzliche Löschpflicht (statt einer Aufbewahrungspflicht) den Vorrang hat.
 - Bekannte technische Grenze, die transparent zu kommunizieren ist: Bereits bestehende **Backups**, die vor dem Löschzeitpunkt erstellt wurden, enthalten das Objekt ggf. noch bis zu ihrem eigenen, separat konfigurierten Ablauf/ihrer eigenen Rotation - vollständige Löschung im produktiven System bedeutet nicht automatisch sofortige Löschung aus jedem historischen Backup-Stand. Das ist eine allgemeine Grenze jeder Lösch-Funktionalität, keine Besonderheit dieses Konzepts, sollte aber im Betriebskonzept (Backup-Rotation) explizit berücksichtigt werden.
 - **Wichtige technische Voraussetzung auf Storage-Ebene** (Zusammenspiel mit 3.6/ 5.1): Wird das WORM-Verhalten aus 5.1 technisch über objektspeicherseitiges Object Lock umgesetzt, muss dafür bewusst der **"Governance"-Modus statt des "Compliance"-Modus** gewählt werden - Compliance-Mode-gesperrte Objekte lassen sich technisch von niemandem, auch nicht mit erweiterten Rechten, vor Fristablauf löschen, was die hier beschriebene Zwangslöschung unmöglich machen würde. Governance-Mode erlaubt eine Löschung durch dafür explizit autorisierte Rollen und bleibt gleichzeitig gegen reguläre, unautorisierte Lösversuche geschützt - welcher Modus für einen Objekttyp greift, sollte daher bewusst mitentschieden werden: Governance-Mode für alles, wofür ggf. eine Zwangslöschung möglich bleiben muss, Compliance-Mode nur für Objekttypen, bei denen das ausgeschlossen ist.
- **Löschgrund**: Ein Freitextfeld/eine Auswahl für den Grund der Löschung kann hinterlegt

werden. Ob die Angabe eines Grundes **optional oder verpflichtend** ist, ist konfigurierbar je Installation/Objektyp - für Installationen mit hohem Dokumentationsbedarf (z. B. öffentliche Verwaltung) empfiehlt sich eine verpflichtende Angabe, während andere Installationen das offener handhaben können. Der Löschrund selbst wird - notwendigerweise getrennt vom gelöschten Objekt - im Audit-Trail (5.3) dauerhaft aufbewahrt, da genau das die Nachvollziehbarkeit *warum* etwas gelöscht wurde erst ermöglicht, auch nachdem der Inhalt selbst nicht mehr existiert.

- **Löschregister:** Jede physische Zwangslöschung wird zusätzlich in einem eigenen, separat gesicherten Löschrund vermerkt (Objekt-ID, Zeitpunkt, Grund) - unabhängig vom regulären Backup-Zyklus der übrigen Systemdaten. Zweck: Nach einer Wiederherstellung aus einem älteren Backup muss zuverlässig geprüft werden können, ob zwischenzeitlich zwangsgelöschte Objekte dabei unbeabsichtigt wiederhergestellt wurden, damit sie erneut gelöscht werden können (siehe 10.4, Backup & Restore).

Einbindung in Prozesse

- Löschrund und die Auslösung der Zwangslöschung lassen sich als Bestandteil einer **BPMN-Prozessdefinition** (7.1) modellieren - z. B. ein automatischer Prozessschritt (Automatic Task), der nach Ablauf der Frist die Löschung anstößt, optional mit vorgeschaltetem Genehmigungsschritt (Manual Task) oder Vier-Augen-Freigabe (4.3) vor der eigentlichen physischen Löschung, falls das für einen Objektyp gewünscht ist.
- Alternativ ist die Zwangslöschung auch unabhängig von einem expliziten BPMN-Prozess direkt am Objekt/Objektyp konfigurierbar - der Prozess-Weg ist eine Option für komplexere Fälle, keine Voraussetzung für die Grundfunktion.

Lösch Erinnerung (optional)

- Optional aktivierbar (Erwartung: eher seltener genutzt, aber unterstützt): Eine konfigurierbare **Vorlaufzeit** vor dem eigentlichen Löschrund löst eine Benachrichtigung über den Notification Service aus, sodass berechtigte Personen die anstehende Löschung noch einmal prüfen und - falls nötig - stoppen oder die Frist anpassen können, bevor die Löschung tatsächlich stattfindet.
- Technisch dieselbe Timer-/Boundary-Event-Mechanik wie bei der SLA-Zeitüberwachung in Prozessschritten (7.1) - kein separater Mechanismus, sondern dieselbe BPMN-Timer-Infrastruktur, nur für einen anderen Zweck genutzt.
- Ohne Reaktion auf die Erinnerung läuft die konfigurierte Löschung nach Ablauf der Frist regulär weiter - die Erinnerung ist eine zusätzliche Kontrollmöglichkeit, kein Vetorecht, das

die Löschung automatisch verhindert.

5.3 Audit-Log

- Wie in 3.4 beschrieben: jedes Event wird unveränderlich, mit Zeitstempel, handelndem Nutzer, betroffenem Objekt und Aktion gespeichert.
- Empfehlenswert: kryptografische Verkettung der Einträge (Hash-Chain), damit nachträgliche Manipulation auch bei DB-Zugriff erkennbar wäre.
- Auswertbar über eigene Admin-UI-Sicht (Audit-Trail je Dokument/Ordner/Nutzer), exportierbar für Prüfungen.

5.4 Reporting & Forensische Nachverfolgung

Der Reporting Service baut auf dem Audit-Event-Strom (3.4/5.3) auf und bedient zwei unterschiedliche Anwendungsfälle:

a) Standardberichte

- Vordefinierte, konfigurierbare Berichte (z. B. Dokumentenaufkommen pro Zeitraum/ Ordner, offene Workflow-Aufgaben, Lizenzauslastung, Speicherverbrauch je Backend, Nutzeraktivität).
- Als Read-Modell umgesetzt: der Reporting Service konsumiert den Event-Strom und hält eigene, für Auswertungen optimierte Aggregationen (siehe 3.1) - kein Live-Join über fremde Schemata.
- Exportierbar (CSV/PDF), planbar (regelmäßiger Versand über den Notification Service).

b) Objektbezogene Nachverfolgung (Forensik-Trace)

- Gezielte Abfrage: „Zeige alle Aktionen, bei denen Objekt X beteiligt war“ - wobei X ein **Nutzer**, ein **Dokument**, ein **Ordner** oder z. B. eine **Rolle** sein kann.
- Zentraler Anwendungsfall: **kompromittierter Account**. Bei Verdacht auf Kompromittierung eines Nutzerkontos lässt sich lückenlos nachvollziehen, welche Aktionen dieser Nutzer **ab einem bestimmten Zeitpunkt** (z. B. dem vermuteten Kompromittierungszeitpunkt) durchgeführt hat - inklusive aller gelesenen, geänderten, heruntergeladenen oder gelöschten Dokumente, Berechtigungsänderungen, Force-Unlocks etc.
- Umsetzung: Da jedes Audit-Event sowohl den handelnden Nutzer als auch alle betroffenen

Objekte referenziert (5.3), kann der Trace beidseitig gefahren werden - "alle Aktionen von Nutzer X" ebenso wie "alle Nutzer, die je auf Dokument Y zugegriffen haben".

- Zeitfenster-Filter (ab/bis) und Aktionstyp-Filter (nur Lesezugriffe, nur Änderungen, nur Downloads, ...) sind Kernbestandteil dieser Ansicht, damit ein Vorfall gezielt eingegrenzt werden kann.
- Ergebnis ist als Bericht exportierbar (z. B. für eine Sicherheitsuntersuchung oder Meldung an Datenschutzbeauftragte) und selbst wieder als Zugriff auditiert (wer hat wann welchen Trace abgefragt).
- Sinnvolle Ergänzung: automatisierte **Anomalie-Hinweise** (z. B. ungewöhnlich hohe Downloadzahl in kurzer Zeit durch einen Nutzer) als optionaler, regelbasierter Vorstufen-Mechanismus zur manuellen forensischen Analyse - kein Ersatz dafür, aber ein Trigger, überhaupt genauer hinzuschauen. **Entscheidung für diese Konzeptversion:** Umsetzung zunächst ausschließlich über konfigurierbare statische Schwellwerte (z. B. "mehr als X Downloads durch denselben Nutzer in Y Minuten"); KI-/ML-gestützte Anomalieerkennung ist bewusst **out of scope** und wird nicht Teil des Kernsystems - eine spätere Ergänzung bleibt durch die "Dazustellen"-Architektur (1., 3.8) jederzeit möglich, ohne dass sie jetzt mitgeplant werden muss.

5.5 Feingranulares User-Tracking (Session-Fingerprinting)

Ergänzend zum regulären Audit-Log (5.3), das primär auf Aktionen bezogen ist, kann pro Nutzer ein **erweitertes Tracking** aktiviert werden, das über die reguläre Aktionsprotokollierung hinausgeht.

- **Standardmäßig nur für den Superuser aktiv** (4.6), für alle anderen Nutzer individuell zuschaltbar (z. B. bei konkretem Verdacht, für besonders privilegierte Konten, oder vertraglich/regulatorisch gefordert für bestimmte Rollen).
- Erfasst zusätzlich zum regulären Audit-Ereignis: vollständige Session-Metadaten (u. a. Client-IP, Geräte-/Browser-Fingerprint, verwendetes Endgerät soweit ermittelbar, Authentifizierungsmethode, Zeitpunkt/Dauer der Session, ggf. bekannte Netzwerk-/Standortinformationen), sodass sich im Bedarfsfall nachvollziehen lässt, von welchem Rechner/Kontext aus ein Konto tatsächlich genutzt wurde - zentral etwa für die Aufklärung, ob und von wem ein Konto kompromittiert oder durch Dritte mitgenutzt wurde.
- Läuft über denselben Audit-/Event-Mechanismus (3.4/5.3), aber mit eigener, feingranularerer Ereignisklasse, damit reguläre Audit-Auswertungen (5.4) nicht standardmäßig mit diesem deutlich größeren Datenvolumen belastet werden.

- Da hierbei besonders sensible personenbezogene Daten anfallen (Geräte-/Standortdaten), gilt ein eigenes, engeres Zugriffsregime auf diese Tracking-Daten (nur wenige, explizit berechnigte Rollen - Zusammenspiel mit 4.6) sowie eine **gesonderte, konfigurierbare Aufbewahrungsdauer (Standardwert: 7 Tage)**, die kürzer bemessen ist als die reguläre Audit-Log-Aufbewahrung, sofern datenschutzrechtlich keine längere Aufbewahrung gerechtfertigt ist (Abwägung analog zu 5.2) - nach Ablauf werden die feingranularen Tracking-Daten automatisch gelöscht, unabhängig vom regulären Audit-Ereignis, das gemäß den allgemeinen Fristen (5.2) bestehen bleibt.
- Aktivierung/Deaktivierung des Trackings für einen Nutzer ist selbst ein auditierter, sensibler Vorgang und kann optional dem Vier-Augen-Prinzip unterliegen (4.3).

5.6 Aussonderung & Langzeitarchivierung

Ergänzend zu den regulären Aufbewahrungsfristen (5.2) unterstützt das System die geregelte **Aussonderung**: die Überführung von Dokumenten/Strukturen, deren aktive Bearbeitungsphase abgeschlossen ist, in ein separates Langzeitarchiv (das selbst wiederum über die Storage-Abstraktion, 3.6, auf einem eigenen, ggf. günstigeren/anders redundanten Backend liegen kann).

- Der Aussonderungsprozess ist technisch eng verwandt mit dem Migrationsprozess (7.2: Sperren → Kopieren/Übertragen → Verifizieren → Freigabe im Archiv → Löschung im aktiven System nach Übergangsfrist) und läuft ebenso als auditierter, resumable Workflow über die Workflow Engine.
- Auslösung konfigurierbar: automatisch nach Ablauf der Aufbewahrungs-/Aktivphase eines Objekttyps, oder manuell angestoßen.
- **Verpflichtende Konvertierung in ein unveränderliches Archivformat**: Vor der Übertragung ins Langzeitarchiv werden Dokumente automatisiert in ein Format konvertiert, das langfristig lesbar und nicht mehr nachträglich unbemerkt veränderbar ist - das Originalformat (z. B. .docx) bleibt dafür nicht ausreichend, da es weiterhin mit dem Ursprungsprogramm bearbeitbar wäre.
 - **PDF/A** als Standardzielformat für einzelne Dokumente - nutzt dieselbe Konvertierungsfunktion, die der Rendering/Preview Service (3.7) für die reguläre PDF/A-Konvertierung ohnehin bereits bereitstellt, hier jedoch verpflichtend statt optional angewendet.
 - **XDOMEA** als Zielformat für strukturierte Akten/Vorgänge (Umlaufmappen, 2.3) - der etablierte deutsche Verwaltungsstandard für den Austausch und die Übergabe von

Akten/Vorgängen an Archive (z. B. Bundesarchiv/Landesarchive), inklusive Aktenstruktur, Metadaten und referenzierter Dokumente in einem einzigen, genormten Austauschformat. Besonders relevant im Public-Sector-Umfeld, wo XDOMEA ohnehin für Übergaben an Archivverwaltungen erwartet wird - stellt die Anschlussfähigkeit an bestehende Archivierungsprozesse der öffentlichen Verwaltung sicher, ohne einen eigenen, inkompatiblen Aussonderungsstandard zu erfinden.

- Bei Umlaufmappen greift diese Konvertierung konsequenterweise auf den Abschluss-Snapshot (2.3) zu - archiviert wird der zum Vorgangsabschluss fixierte Stand, nicht ein womöglich zwischenzeitlich weiterverändertes Original.
- Das Originalformat (z. B. die ursprüngliche .docx-Version) bleibt zusätzlich zur Archivformat-Konvertierung erhalten (keine Ersetzung), sofern nicht separat anders konfiguriert - die Konvertierung ergänzt die Archivierung, statt Information zu verlieren.
- **Optionale Verschlüsselung vor der Aussonderung:** Dokumente können vor der Übertragung ins Langzeitarchiv mit einem eigenen, vom laufenden Betrieb getrennten Schlüssel verschlüsselt werden, sodass ein Datenleck im Archiv (das i. d. R. seltener geprüft/gewartet wird als das aktive System) nicht unmittelbar zu lesbaren Inhalten führt.
 - **Schlüsselverwaltung erfolgt bewusst extern**, nicht im System selbst: die zur Ver-/Entschlüsselung nötigen Schlüssel werden nicht innerhalb der DMS-Datenbank oder eines DMS-eigenen Schemas gespeichert, sondern in einem separaten, dafür etablierten Schlüsselverwaltungsformat - z. B. einer **KDBX-Datei** (KeePass-Datenbankformat) - die eigenständig gesichert, gesichert aufbewahrt und mit eigenem Master-Kennwort/Zugriffsschutz versehen wird. Damit ist ein kompromittiertes DMS/Archiv allein nicht ausreichend, um an die Schlüssel zu gelangen, da diese in einem komplett getrennten Aufbewahrungsort liegen. Größere Installationen können statt der Datei-Variante auch ein vollwertiges externes KMS/HSM anbinden - die KDBX-Variante ist der einfache, ressourcenschonende Standardweg, kein Zwang.
 - Konfigurierbar pro Objekttyp/Mandant, ob und mit welchem Schema verschlüsselt wird (z. B. je Dokument individueller Schlüssel vs. gemeinsamer Archiv-Schlüssel je Zeitraum - Abwägung zwischen Aufwand und Isolationsgrad im Schadensfall).
- Ausgesonderte Dokumente bleiben über Metadaten und Audit-Trail weiterhin auffindbar/nachvollziehbar (Verweis im aktiven System, wo sich das Original befindet), auch wenn der Inhalt selbst nicht mehr im aktiven Storage-Zielsatz liegt - wichtig für Auskunftsfähigkeit trotz Auslagerung. Während der Übergangsfrist (7.2) bleiben ausgesonderte Elemente

zusätzlich direkt im eigenen **Aussonderung-Sonderbereich** (2.5) durchsuch-/einsehbar, nicht nur indirekt über Audit-Trail-Verweise.

- Rückholung aus dem Archiv (falls doch wieder benötigt) ist ein eigener, ebenfalls audierter Vorgang mit Entschlüsselung nur für berechtigte Rollen.

6. Diagnose & Systemwerkzeuge

6.1 Zentrale Query- & Trace-Konsole

Eine zentrale, mächtige Abfrageschnittstelle über das gesamte System - gedacht sowohl für Fehleranalyse/Support als auch dafür, dass Administrierende oder Integratoren eigene Monitoring-Sensoren (10.1) oder Auswertungen selbst formulieren können, ohne auf vordefinierte Berichte (5.4a) beschränkt zu sein.

Grundprinzip

- Eigener **Query & Trace Service**, der über den Event-Bus (3.4) und die Read-Modelle der übrigen Services (Audit, Reporting, Monitoring) eine einheitliche Abfrageschicht bereitstellt - kein Direktzugriff auf fremde DB-Schemata (Prinzip aus 3.1 bleibt gewahrt), sondern eine föderierte Sicht über die jeweils dafür vorgesehenen Schnittstellen/Read-Modelle der Owner-Services.
- Abfragesprache **bewusst mächtig und einheitlich für Lese- und Schreibfälle** - das ist so gewollt und kein Versehen: Die Mächtigkeit ist Voraussetzung dafür, dass die Konsole tatsächlich als vollwertiges Diagnose-/Debugging-Werkzeug taugt, nicht nur als eingeschränktes Reporting-Frontend. Syntaktisch orientiert sich die Sprache konkret **am SQL-Dialekt von PostgreSQL (psql)** - naheliegend, da Postgres ohnehin die zugrunde liegende Datenbank ist (3.1); Administrierende, die mit `psql` vertraut sind, finden sich direkt zurecht, statt eine komplett eigene Notation lernen zu müssen. Technisch wird dafür bewusst **kein eigener Parser von Grund auf geschrieben**, sondern der echte PostgreSQL-Parser wiederverwendet und erweitert (konkret über die Python-Bibliothek `pghost/libpg_query`, siehe 1a) - die DMS-spezifischen Zusatzkonstrukte für Trace-/Hierarchie-/Ereignisabfragen (z. B. Traversierung von Ordnerhierarchien, Event-Zeitfenster-Operatoren) werden als Erweiterung des daraus gewonnenen AST ergänzt, statt PostgreSQL-Grammatik nachzubilden. Damit lassen sich sowohl beliebige Trace-

/Auswertungsfragen ("alle Ereignisse zu Dokument X der letzten 90 Tage, gruppiert nach Nutzer") als auch gezielte Dateneingriffe formulieren ("setze Attribut Y auf allen Dokumenten vom Typ Z mit Bedingung B zurück"). Die Sprache ist damit auch die Grundlage, um **eigene, wiederverwendbare Sensor-Definitionen** für das Monitoring (10.1) zu formulieren, statt dafür jeweils Code schreiben zu müssen.

- **Funktionsumfang strikt an die Rechte der ausführenden Person gekoppelt - ohne Authentifizierung ist keine einzige Abfrage möglich** (kein anonymer/technischer Zugriff, auch nicht rein lesend): Die Konsole führt jede Abfrage im Sicherheitskontext des eingeloggten Nutzers aus und wendet dabei exakt dieselben RBAC-Rechte, Ordnerberechtigungen und Bereichssperren (4.7) an, die auch sonst für diese Person gelten - eine Query kann nie mehr sehen oder verändern, als die ausführende Person ohnehin dürfte. Der aktivierte Superuser (4.6) ist die einzige Ausnahme und kann über die Konsole tatsächlich uneingeschränkt lesen und schreiben.
- Für die Manipulationsseite gelten zusätzlich zur Rechtebindung verpflichtend mehrere Sicherungsstufen, die kumulativ gelten (nicht optional einzeln abschaltbar in Kombination, sondern jede Stufe für sich konfigurierbar):
 - i. **Schutzschalter ("Manipulationsmodus")**: Schreibende Abfragen sind standardmäßig deaktiviert und müssen explizit für eine begrenzte Zeit/Sitzung durch eine berechtigte Rolle freigeschaltet werden (vergleichbar einem "Break-Glass"-Zugang, siehe auch 4.6). Ohne aktiven Schutzschalter werden schreibende Ausdrücke bereits beim Parsen abgelehnt, nicht erst bei der Ausführung.
 - ii. **Dry-Run als Standardverhalten**: Jede schreibende Abfrage wird zunächst nur simuliert - das Ergebnis zeigt, wie viele Objekte betroffen wären und eine Vorschau der Änderungen, bevor eine zweite, explizite Bestätigung die tatsächliche Ausführung auslöst.
 - iii. **Optionales Vier-Augen-Prinzip** (konsistent mit 4.3): Manipulierende Abfragen können - konfigurierbar je Installation - eine zweite Freigabe durch eine andere berechtigte Person erfordern, bevor sie nach dem Dry-Run tatsächlich ausgeführt werden.
 - iv. **Verpflichtendes Vier-Augen-Prinzip für als kritisch markierte Tabellen/Objektbereiche**: Unabhängig von der allgemeinen, installationsweiten Vier-Augen-Konfiguration aus Punkt 3 können einzelne besonders sensible Tabellen/Entitätsklassen (z. B. Berechtigungs-/Rollentabellen, Lizenzdaten, Objekttyp-/Constraint-Definitionen) als **zwingend vier-Augen-pflichtig** markiert werden - diese Markierung lässt sich nicht durch eine abweichende allgemeine Konfiguration

umgehen und gilt **auch für den aktivierten Superuser**, der sonst über die Konsole uneingeschränkt agieren kann. Damit bleibt selbst der mächtigste Zugriffsweg für die wirklich kritischsten Datenbereiche zusätzlich abgesichert.

- v. **Vollständige Protokollierung:** Jede Abfrage (lesend wie schreibend), der ausgeführte Query-Text, das Ergebnis/die Auswirkung und die beteiligten Personen (Ausführende, ggf. Vier-Augen-Freigebende) werden auditiert (5.3) - die Konsole selbst unterliegt also derselben Nachvollziehbarkeit, die sie für den Rest des Systems bereitstellt.
- Zugriff auf die Konsole ist selbst RBAC-gesteuert und granular einschränkbar (z. B. "nur lesend", "nur auf Domäne X", "keine Manipulation") - passt zum Konzept der abgespeckten, domänengetrennten Admin-Rollen (4.6).
- Die Konsole ist sowohl über eine UI-Komponente (Bestandteil der Admin-Weboberfläche, 8) als auch über das CLI-Tool (6.2) nutzbar - dieselbe Abfragesprache und dieselben Sicherungsstufen gelten in beiden Zugängen gleichermaßen.

6.2 CLI-Tool

Ein vollständiges Kommandozeilenwerkzeug für Konfiguration, Auditing, Tracing und Debugging - konzeptionell an Werkzeugen wie `oc` (OpenShift) orientiert: ein Client, der gegen die Kern-APIs des Systems spricht (über das API-Gateway/BFF, 3.5) und damit dieselben Berechtigungen und Sicherungsstufen respektiert wie die Web-UIs.

- **Funktionsumfang:** Konfigurationsim-/export (7.3), Abfragen über die Query & Trace Konsole (6.1), Auslösen/Beobachten von Migrations-/Transfer-Vorgängen (7.2), Verwaltung von Objekttypen/Constraints/Workflows, Einsicht in Registry-/Plugin-Orchestrierungsstatus (3.2/3.8), Lizenzstatus (9.3), Nutzer-/Rollenverwaltung, Auslösen/Überwachen von Backup- und Restore-Vorgängen inkl. Fortschritt des Löscharbeits (10.4), Delta-/Vergleichsläufe zwischen Installationen (7.5) mit skriptfähiger Ausgabe.
- **Domänentrennung:** Das CLI ist modular aufgebaut, sodass es (wie vorgeschlagen) in fachliche Teilkommandos gegliedert werden kann, die getrennt ausgerollt/berechtigt werden können - z. B. ein schlankeres CLI-Subset nur für Auditing/Reporting an Fachbereiche, während administrative Eingriffe (Objekttypen, Migration, Manipulation über 6.1) einer engeren Nutzergruppe vorbehalten bleiben.
- Authentifizierung über dieselben Mechanismen wie die Web-UIs (4.4), inkl. Unterstützung für Service-Accounts (z. B. für CI/CD-Pipelines, die Konfiguration automatisiert ausrollen).
- Scriptbar/automatisierbar (strukturierte Ausgabeformate wie JSON), damit sich CLI-Aufrufe in Automatisierungs- und Deployment-Pipelines einbetten lassen (z. B. automatisiertes

7. Prozesse: Workflow Engine, Migration, Konfiguration

7.1 BPMN-basierte Workflow Engine

Die Workflow Engine basiert auf **BPMN 2.0** (Business Process Model and Notation) als Modellierungs- und Ausführungsstandard, statt eines proprietären Zustandsautomaten-Formats - das erlaubt sowohl die Modellierung eigener Prozesse direkt im System als auch den **Import bestehender BPMN-Modelle**, die z. B. mit externen Modellierungswerkzeugen erstellt wurden. Konkrete Umsetzung auf Basis von **SpiffWorkflow** (siehe 1a) als Python-native BPMN-Ausführungsbibliothek.

Modellierung & Import

- Ein **Process Designer** (eigenständige Frontend-Anwendung, **nicht** Bestandteil der Admin-Weboberfläche, siehe 8) erlaubt die grafische Modellierung von Geschäftsgängen/Prozessen direkt im System - Ergebnis ist ein Standard-BPMN-2.0-XML-Dokument.
- **Import bestehender BPMN-Dateien** (BPMN 2.0 XML) aus externen Werkzeugen wird unterstützt; beim Import erfolgt eine Validierung gegen die im System verfügbaren Task-Typen/Konnektoren (z. B. ob referenzierte Objekttypen, Ordnerziele oder Instanz-Ziele bei föderierten Schritten tatsächlich existieren) mit klarer Fehlermeldung bei nicht auflösbaren Referenzen.
- Prozessdefinitionen (importiert oder selbst modelliert) sind Teil der JSON-/BPMN-Konfiguration und über den Konfigurationsexport (7.3) exportier-/importierbar - passt zum bestehenden Cloning-/Übertragungsprinzip zwischen Installationen.

Geschäftsobjekte durch Prozesse führen

- Ein BPMN-Prozess führt **Geschäftsobjekte** - Dokumente oder Umlaufmappen (2.3) - automatisiert durch die im Prozess definierten Abteilungen/Rollen/Bearbeitungsschritte, inklusive der zugehörigen Statusübergänge, Berechtigungsprüfungen (RBAC, 4.1) und Constraint-Prüfungen (2.2) an jedem Schritt.

- **Manual Tasks (User Tasks):** erfordern eine aktive Handlung einer berechtigten Person (Freigabe, Prüfung, Bearbeitung) - erscheinen als Aufgabe in der User-/Reviewer-UI (8).
- **Automatic Tasks (Service Tasks):** werden vom System selbst ausgeführt, ohne manuelles Zutun (z. B. automatische Weiterleitung nach Fristablauf, automatisierter Constraint-Check, Auslösen eines Connector-Aufrufs) - Standard-BPMN-Konzept, direkt unterstützt.
- **Signature Tasks:** dritter Task-Typ, der eine elektronische Signatur (3.10) einer berechtigten Person am jeweiligen Geschäftsobjekt verlangt, bevor der Prozess fortgesetzt wird - technisch eine Sonderform des Manual Tasks, fachlich aber eigenständig modellierbar, da das erforderliche Signaturniveau (SES/AES/QES) je Prozessschritt festgelegt wird.

Zeitüberwachung je Prozessschritt (SLA-Tracking)

- Für jeden einzelnen Prozessschritt lässt sich eine **maximale zulässige Bearbeitungsdauer** konfigurieren (Teil der Prozessdefinition, nicht global für den ganzen Prozess).
- Überschreitet ein Schritt diese Dauer, löst das System eine konfigurierbare Eskalation aus (z. B. Benachrichtigung an eine Vorgesetzten-Rolle über den Notification Service, Markierung als überfällig in der UI, optional automatische Eskalationsverzweigung im BPMN-Modell selbst über einen Boundary-Timer-Event) - Ziel ist, dass kein Vorgang unbemerkt liegen bleibt.
- Die tatsächliche Bearbeitungsdauer je Schritt wird als Audit-Ereignis erfasst (5.3) und ist damit auch Grundlage für Standardberichte (5.4a) und die Forensik-/Trace-Konsole (6.1) - z. B. "durchschnittliche Bearbeitungsdauer je Schritt/Abteilung" als Reporting-Kennzahl.

Multi-Instanz-Prozesse (installationsübergreifende Prozessschritte)

- Ein Prozessschritt kann als **Übergabe an eine oder mehrere andere Installationen** definiert werden - technisch über den Federation Hub (7.4) abgewickelt, das Geschäftsobjekt (Dokument/Umlaufmappe) wird an die im Prozessschritt hinterlegte(n) Zielinstallation(en) übergeben.
- **Das Ziel/die Ziele werden bei der Prozessmodellierung fest in den Prozessschritt eingetragen** (keine Laufzeit-Auswahl durch die bearbeitende Person) - die zur Auswahl stehenden Installationen stammen dabei aus dem **Adressbuch des Federation Hub** (7.4).
- **Diese Option steht nur zur Verfügung, wenn die Installation tatsächlich an einem Federation Hub teilnimmt** (7.4 ist optional) - ist kein Hub konfiguriert/kein Adressbuch vorhanden, bietet der Process Designer föderierte Prozessschritte gar nicht erst als Auswahlmöglichkeit an, statt eine Option anzuzeigen, die ins Leere liefe.
- Nach Übergabe wird der Prozessfortschritt auf beiden Seiten (abgebende wie

empfangende Installation) jeweils lokal weiter nachvollzogen; die Rückmeldung des Bearbeitungsergebnisses läuft wieder über denselben Federation-Hub-Mechanismus zurück an den ursprünglichen Prozess, sofern der Prozess das vorsieht (z. B. bei einer Prüfung durch eine andere Behörde mit anschließender Rückgabe).

7.2 Migration/Transfer (Ex- und Import von Strukturen)

Übernahme des vorgeschlagenen Prozesses, ergänzt um Absicherungen:

1. **Sperre** des Quellordners (inkl. aller Unterordner) - keine Schreibzugriffe während der Migration.
2. **Kopieren** der Inhalte inkl. Metadaten, Versionshistorie, Berechtigungen, Audit-Referenzen in das Zielsystem.
3. **Verifizierung**: Prüfsummen-/Hash-Vergleich pro Dokument (nicht nur Anzahl-Abgleich), Abgleich der Metadaten- und Versionsvollständigkeit.
4. **Freigabe im Zielsystem** nach erfolgreicher Verifizierung.
5. **Löschung im Quellsystem** erst nach Ablauf einer konfigurierbaren Übergangs-/Aufbewahrungsfrist (Sicherheitsnetz für Rollback).

Ergänzend:

- Der gesamte Vorgang läuft selbst als auditierbarer, resumable Workflow über die Workflow Engine (nicht als Sonderfall daneben) - bei Abbruch ist ein Fortsetzen ab dem letzten bestätigten Schritt möglich, statt neu beginnen zu müssen.
- **Dry-Run-Modus**: Simulation des Transfers (inkl. Constraint-Prüfung im Zielsystem, z. B. passende Objekttypen vorhanden?) ohne tatsächliche Datenbewegung.
- Migration/Transfer ist eine eigene, optional lizenzierbare Komponente und kann selbst dem Vier-Augen-Prinzip unterliegen (siehe 4.3).

7.3 Konfigurationsimport/-export (JSON)

- Vollständige Systemkonfiguration (Objekttypen, Constraints, Workflows, Rollen-/Rechte-Templates, Vier-Augen-Einstellungen je Aktionstyp, UI-Anpassungen) exportierbar als JSON-Dokument.
- Re-Import in ein anderes (oder dasselbe, z. B. Staging→Produktion) System möglich - damit auch Basis für Systemklonierung/Testumgebungen.
- Versionierung des Konfigurationsschemas selbst, damit Export aus einer älteren Version in

eine neuere importiert werden kann (mit Migrationslogik bei Schemaänderungen).

7.4 Mandantenübergreifende Workflows (Föderations-Drehscheibe)

Ergänzend zur reinen Migration (7.2, die einen einmaligen, endgültigen Transfer beschreibt) unterstützt das System **laufende Geschäftsprozesse, die über die Grenzen vollständig unabhängiger Installationen hinweg funktionieren** - z. B. ein Freigabeschritt, der bei einem Partnerunternehmen mit eigener Installation liegt, oder ein Dokument, das im Rahmen eines Prozesses zwischen mehreren Organisationen wandert, ohne dass eine der beteiligten Installationen die Kontrolle über die jeweils andere hat.

Föderations-Drehscheibe als eigener, separater Baustein

- Ein dedizierter **Federation Hub Service** (bewusst kein Teil einer einzelnen Installation, sondern ein eigenständig betriebener, ggf. von einem neutralen Betreiber bereitgestellter Vermittlungsdienst) übernimmt zwei Aufgaben, analog zum Registry-Prinzip (3.2) aber installationsübergreifend:
 - **Adressbuch**: Installationen melden sich beim Hub mit einer öffentlichen Kennung und den Prozess-/Dokumenttypen an, für die sie als Teilnehmer an föderierten Workflows zur Verfügung stehen - keine Installation muss die internen Adressen/Endpunkte einer anderen direkt kennen.
 - **Schaltzentrale**: Der Hub vermittelt Prozessschritte/Dokumentübergaben zwischen mehr als zwei teilnehmenden Installationen, ohne selbst Dokumenteninhalte dauerhaft zu speichern - er leitet Übergaben weiter und protokolliert nur Metadaten des Vermittlungsvorgangs (wer hat wann was an wen übergeben), nicht die Dokumentinhalte selbst.
- Jede Installation bleibt bezüglich ihrer eigenen Daten vollständig autonom (Prinzip aus 3a bleibt gewahrt) - ein föderierter Workflow-Schritt ist aus Sicht jeder beteiligten Installation ein regulärer, lokal auditierter Ein-/Ausgang (vergleichbar einem Poststelleneingang von außen), nicht ein Fremdzugriff auf die eigene Datenbank.
- Föderierte Workflow-Schritte sind Teil der normalen Workflow-Engine-Definition (7.1): ein Prozess kann einen Schritt enthalten, der "Übergabe an externe Installation via Federation Hub" bedeutet, inkl. Wartezustand bis zur Rückmeldung.

Betreibermodell

- Da die Software Open Source ist, sind **beide Betriebsvarianten des Federation Hub explizit vorgesehen und gleichwertig unterstützt**: Ein Kunde/Betreiber kann seinen eigenen Federation Hub selbst betreiben (z. B. für die Föderation zwischen den eigenen mehreren Installationen, 3a), oder ein Betreiber stellt einen Hub auch für andere, fremde Installationen bereit (z. B. ein Verbund mehrerer Organisationen, die sich auf einen gemeinsam betriebenen Hub einigen).
- Die Entwicklerseite/das Projekt selbst betreibt **keinen öffentlichen produktiven Federation Hub** als eigenes Angebot - ein zentral vom Projekt bereitgestellter Hub existiert allenfalls für interne Zwecke oder Testing, nicht als Bestandteil des Produkts für Endkunden. Das ist eine bewusste Konsistenzentscheidung zum Open-Source-Charakter: die Föderation bleibt vollständig in der Hand der Betreiber, keine zentrale Abhängigkeit von einer Projekt-Infrastruktur.

Versionskompatibilität zwischen Installationen

- Da an einer Föderation Installationen mit unterschiedlichem Versionsstand teilnehmen können, meldet jede Installation dem Hub bei der Registrierung nicht nur ihre eigene Version, sondern auch eine **erklärte Kompatibilitätsspanne** - z. B. "ich bin Version 9.3, kann aber auch mit Versionen ab 8.2 föderierte Workflows abwickeln".
- Der Hub prüft bei der Vermittlung eines föderierten Prozessschritts, ob sich die Kompatibilitätsspannen der beteiligten Installationen überschneiden, und lehnt die Vermittlung mit klarer Fehlermeldung ab, falls nicht - statt einen Schritt zu vermitteln, den eine Seite gar nicht korrekt verarbeiten kann.
- Diese Kompatibilitätsspanne ist Teil des ohnehin schon versionierten Konfigurationsschemas (7.3) und wird mit jedem Release des Systems explizit gepflegt (welche älteren Versionen bleiben föderationskompatibel), nicht implizit angenommen.

Sicherheit & Governance

- Teilnahme am Hub ist opt-in je Installation und je Prozess-/Dokumenttyp - eine Installation legt explizit fest, mit welchen anderen (bekannten, vertrauenswürdigen) Installationen sie föderierte Prozesse führt, kein automatisches offenes Netzwerk.
- Übertragene Dokumente/Daten werden Ende-zu-Ende zwischen den beteiligten Installationen verschlüsselt (der Hub selbst kann die Inhalte nicht einsehen, nur vermitteln) - technisch verwandt mit der optionalen Verschlüsselung bei der Aussonderung (5.6).
- Jede Übergabe ist auf beiden beteiligten Seiten vollständig auditiert (5.3); zusätzlich kann - wie bei der regulären Migration - eine Verifizierung (Prüfsummen) vor endgültigem

Abschluss des Übergabeschritts erfolgen.

7.5 Delta-/Vergleichsfunktion zwischen Installationen

Ergänzend zum reinen Konfigurationsexport/-import (7.3) unterstützt das System einen gezielten **Vergleich zweier Installationen** gegeneinander - gedacht z. B. für Drift-Erkennung zwischen Staging und Produktion, für den Abgleich mehrerer vom selben Betreiber verwalteter Installationen (3a), oder um vor einer geplanten Konfigurationsübernahme zu sehen, was sich tatsächlich unterscheiden würde.

Grundprinzip: Basisinstanz vs. Vergleichsinstanz

- Eine Installation wird als **Basisinstanz** (Referenz) festgelegt, eine oder mehrere andere werden **dagegen verglichen**. Das Ergebnis ist ein gerichteter Delta-Bericht ("was weicht in der Vergleichsinstanz von der Basisinstanz ab"), nicht nur eine ungerichtete Ähnlichkeitsprüfung.
- Der Vergleich arbeitet auf Basis der bestehenden **Konfigurationsexporte** (7.3) beider Installationen, nicht über einen direkten Live-Zugriff einer Installation auf die Datenbank einer anderen - das bleibt konsistent mit dem Grundsatz vollständiger Installationsisolation (3a). Die beiden Exporte können manuell bereitgestellt oder - falls beide Installationen an einem gemeinsamen Federation Hub (7.4) teilnehmen - über den dortigen kontrollierten Kanal automatisiert abgerufen werden; ein Hub ist aber keine Voraussetzung für die Funktion.
- Die Funktion ist **rein lesend/diagnostisch**: Sie verändert nichts an einer der beiden Installationen. Sollen erkannte Unterschiede tatsächlich übernommen werden, geschieht das bewusst über den regulären, bereits abgesicherten Konfigurationsimport (7.3) - mit allen dort geltenden Schutzmechanismen (z. B. Vier-Augen, falls konfiguriert) -, nicht über einen eigenen, ungeprüften Synchronisationspfad.

Konfigurierbarer Vergleichsumfang (Default: alles)

- Analog zum bereits etablierten Muster beim Monitoring (10.1: "alles oder nichts, granular einstellbar") gilt hier: Der Vergleich deckt standardmäßig **alle** vergleichbaren Kategorien ab - Objektdefinitionen (2.2), BPMN-Prozesse (7.1), Rollen-/Berechtigungs-Templates (4.1), Vier-Augen-Konfiguration je Aktionstyp (4.3), AD-Gruppen-Mapping-Regeln (4.4), Constraint-Definitionen, UI-Anpassungen, Monitoring-Sensor-Konfiguration (10.1) - und lässt sich gezielt auf einzelne Kategorien eingrenzen, wenn nur ein Teilbereich interessiert

(z. B. "nur Objektdefinitionen und Prozesse vergleichen, Berechtigungen ignorieren").

- Rein installationsspezifische Daten (z. B. Lizenzstand, 9.1, oder die konkrete Registry-Erreichbarkeit, 3.2) sind vom Vergleich grundsätzlich ausgenommen, da ein Unterschied hier keine Konfigurationsabweichung, sondern erwartetes, installationsspezifisches Verhalten darstellt.

Konfigurierbare Ignore-Regex für Objektnamen

- Da Objekte (Objekttypen, Prozesse, Rollen etc.) zwischen Installationen abweichende Namenskonventionen haben können, ohne dass das inhaltlich einen echten Unterschied darstellt - genau das im Beispiel genannte Szenario, in dem `100_testobjekt_typ_alpha` auf der einen und `101__testobjekt_typ_alpha` auf der anderen Installation eigentlich dasselbe Objekt meinen -, lässt sich eine **Ignore-Regex** definieren, die vor dem eigentlichen Namensabgleich auf beide Namen angewendet wird (Normalisierung/Bereinigung, z. B. Entfernen numerischer Präfixe und uneinheitlicher Trennzeichen), sodass die bereinigten Namen als identisch erkannt werden, auch wenn die Rohwerte unterschiedlich sind.
- Diese Ignore-Regel betrifft **ausschließlich die Zuordnung, welches Objekt auf beiden Seiten als "dasselbe" gilt** (die Objekt-Identifikation für den Abgleich) - der eigentliche inhaltliche Vergleich der übrigen Attribute des so zugeordneten Objekts (Constraints, Berechtigungen, Feldstruktur etc.) erfolgt davon unabhängig weiterhin vollständig, damit inhaltliche Abweichungen trotz "gleichem" Namen nicht unter den Tisch fallen.
- Konfigurierbar sowohl **global** (eine Ignore-Regex für alle Kategorien) als auch **je Kategorie** (z. B. unterschiedliche Namenskonventionen bei Objekttypen vs. Prozessen), da sich Namensschemata zwischen Objektarten typischerweise unterscheiden.
- Ohne konfigurierte Ignore-Regex gilt der exakte Namensvergleich als Standardverhalten - die Funktion ist also ohne jede Konfiguration bereits nutzbar, die Ignore-Regex ist eine optionale Verfeinerung.

Ergebnisdarstellung

- Delta-Bericht mit klarer Kategorisierung: **nur in Basisinstanz vorhanden** (fehlt in der Vergleichsinstanz), **nur in Vergleichsinstanz vorhanden** (zusätzlich, nicht in der Basis), **in beiden vorhanden, aber inhaltlich abweichend** (mit Detailanzeige, welche Attribute konkret abweichen), **identisch** (der überwiegende Normalfall, üblicherweise eingeklappt/zusammengefasst dargestellt statt jedes identische Objekt einzeln aufzulisten).
- Verfügbar sowohl über die Admin-UI (8) als visuelle Aufbereitung als auch über das CLI-

Tool (6.2) in strukturierter, skriptfähiger Ausgabe - relevant z. B. für automatisierte Drift-Checks in einer CI/CD-Pipeline vor einem geplanten Konfigurationsrollout auf mehrere Installationen.

8. UI-Schicht

- **User-Weboberfläche:** Dokumenten-/Ordernavigation (inkl. Ordner anlegen/umbenennen/verschieben, nicht nur Durchsuchen), Metadaten-Anzeige/-Bearbeitung je Dokument (Attribute gemäß Objekttyp, 2.2/4.5 - die Objekttyp-Definition selbst bleibt Sache der Admin-UI, ihre Werte am konkreten Dokument gehören in die User-UI), Suche, Upload, Workflow-Interaktion (Freigaben, Aufgaben), Vorschau.
 - **Layout-Leitbild (Standardanordnung):** dreigeteilter Hauptbereich, angelehnt an bekannte Desktop-Dateimanager-Metaphern (Wiedererkennbarkeit senkt die Einstiegshürde für Endnutzer, die einen solchen Umgang bereits aus dem Betriebssystem kennen) - links der Explorer, oben rechts die Dokumenttabs (mehrere geöffnete Dokumente/Ordner parallel, analog zu Browser-/Editor-Tabs) über der Dokumentvorschau (2.4/3.7), darunter das Metadaten-Panel des im aktiven Tab ausgewählten Dokuments. Die Tab-Auswahl steuert Metadaten-Panel und Vorschau synchron - wechselt der Tab, wechseln beide gemeinsam. Diese Anordnung ist der **Werksstandard**, keine feste Vorgabe (siehe flexibler Arbeitsbereich unten).
 - **Flexibler, anpassbarer Arbeitsbereich (Editor-Metapher, analog VS Code):** Über die Standardanordnung hinaus lässt sich der Hauptarbeitsbereich frei umbauen - Panels (Explorer, Metadaten, Vorschau, weitere Dokumenttabs) können nebeneinander oder übereinander angeordnet, in eigene Bereiche abgespalten oder wieder zusammengeführt werden, vergleichbar dem dockbaren Editor-/Panel-Layout von Visual Studio Code. Insbesondere lassen sich mehrere Dokumente gleichzeitig sichtbar nebeneinander oder übereinander platzieren, statt sie nur nacheinander über Tabs anzuzeigen. Die Anpassung ist eine reine Anzeigepräferenz (analog zur Explorer-Baumansicht, 2.2a/8) und wird **je Gerät lokal** gemerkt, nicht geräteübergreifend wie das Farbschema (unten) - ein Arbeitsbereich-Layout ist typischerweise an die Bildschirmgröße/den Nutzungskontext des jeweiligen Geräts gebunden. Ein "Zurücksetzen auf Standardanordnung" ist jederzeit möglich.
 - **Bewusst zurückgestellt: freie CSS-/Themeanpassung durch Endnutzer** - über die

vier Farbschemata (unten) hinaus wird den Nutzenden keine eigene CSS-Anpassung angeboten. Technisch wäre das ohne großen Mehraufwand möglich (dieselbe Grundlage wie die Arbeitsbereich-Flexibilität oben), birgt aber das Risiko, unerfahrene Nutzende mit zu vielen Freiheitsgraden zu überfordern oder zu inkonsistent wirkenden, schwer supportbaren Oberflächen zu führen - bleibt daher ein späterer, unverbindlicher Ausbaukandidat statt Teil dieser Konzeptversion.

- Ganz am linken Rand zusätzlich eine schmale, iconbasierte Navigationsleiste für Cross-Cutting-Funktionen (z. B. Aufgaben/Postkorb, Suche, Profil/Einstellungen) - bewusst außerhalb des dreigeteilten Haupt-Workspace, da sie nicht zum eigentlichen Dokumenten-Arbeitsbereich gehört.
- **Explorer-Darstellung wählbar:** Neben der klassischen Listenansicht kann die Nutzerin/der Nutzer den Explorer auf eine **Baumansicht** umschalten, die die tatsächliche Ordnerhierarchie strukturell abbildet - eine reine Anzeigepräferenz (analog zum Farbschema unten), keine Systemvorgabe; welche Darstellung genutzt wird, entscheidet jede Person für sich. In beiden Ansichten wird vor dem Namen jedes Ordners/Dokuments das **Klassen-Icon** (2.2a/2.2b) der jeweiligen Objekttyp-Zuordnung angezeigt, damit sich unterschiedliche Klassen auf einen Blick unterscheiden lassen, ohne den Objekttyp-Namen einblenden zu müssen.
- **Admin-Weboberfläche:** Nutzer-/Rollenverwaltung, Objekttyp-/Constraint-Editor, Lizenzübersicht, Audit-Trail-Ansicht, Konfigurationsim-/export, Registry-/Service-Übersicht (welche Services sind aktiv, lizenziert, Health-Status). Der Workflow-/Process-Designer ist **nicht** Teil der Admin-UI (siehe eigener Punkt unten).
 - **Layout-Leitbild:** klassisches Management-Dashboard - linke, ggf. ausklapp-/gruppierbare Navigationsseitenleiste (Bereiche wie Nutzer & Rollen, Objekttypen, Registry, ...; weitere Untergliederung, sobald mehr Bereiche dazukommen), Hauptbereich rechts zeigt jeweils die gewählte Funktion.
 - **Objekttyp-/Layout-Editor als geführte GUI, nicht als Text-/JSON-Eingabe:** Der Objekttyp-Editor (2.2/2.2a/2.2b) ist ein formularbasierter Assistent - die administrierende Person wählt Attribute aus, vergibt je Attribut einen Anzeigenamen und markiert Pflichtfelder, statt die zugrunde liegende JSON-Struktur (2.2) von Hand zu editieren. Aus diesen Angaben wird automatisch ein Standardlayout ("Smart Layout") erzeugt. Ein ergänzender **Layout-Designer** (eigener Admin-UI-Bereich) erlaubt das gezielte Nachjustieren dieses generierten Layouts - Spalten-/Zeilenanordnung der Felder - sowohl für die Metadaten-Anzeige als auch für die davon unabhängig anpassbaren Such- und Uploadmasken desselben Objekttyps

(2.2b).

- **Mehrere Installationen aus einer Admin-UI heraus verwalten:** Da eine Installation (3a) ihre eigene, vollständig isolierte Identität besitzt, hält die Admin-UI eine Liste konfigurierter Installationen (je mit eigenem Gateway-Endpunkt) und eine eigene Sitzung je Installation - ein Wechsel zwischen bereits einmal angemeldeten Installationen erfordert keine erneute Anmeldung, solange die jeweilige Sitzung gültig ist. Kein Single-Sign-on über Installationsgrenzen hinweg (widersprüche der bewussten Isolation aus 3a), aber auch keine Notwendigkeit, die Admin-UI mehrfach zu betreiben oder sich pro Installation jedes Mal neu anzumelden.
- Der **Process Designer** (BPMN-Modellierung, 7.1) ist eine **eigenständige Frontend-Anwendung**, nicht Bestandteil der Admin-Weboberfläche - eigener fachlicher Fokus (Prozessmodellierung statt Verwaltung), eigener technischer Bedarf (BPMN-Modellierungskomponente, z. B. `bpmn-js`, siehe 1a), unabhängig von den übrigen UIs deploybar.
- Perspektivisch denkbar: dedizierte **Reviewer/Approval-UI** (schlanker Fokus nur auf Freigabeaufgaben, auch für Vier-Augen-Fälle) sowie eine **Migrations-Konsole** für Transfer-Vorgänge.
- **Farbschema:** Jede Web-UI unterstützt **hell, dunkel, hoher Kontrast** (Barrierefreiheit) und **automatisch** (folgt der Systemeinstellung des Geräts). Die Präferenz wird **im Nutzerprofil hinterlegt**, nicht nur lokal im Browser, und wirkt sich **geräteübergreifend** aus - meldet sich dieselbe Person an einem anderen Gerät an, gilt dieselbe Einstellung dort ebenfalls.
- Anpassbarkeit: konfigurierbare Sichten/Formulare pro Objekttyp (welche Attribute werden wo angezeigt/editierbar), Branding/Theming, ggf. rollenabhängige Dashboards.
- Technisch als eigene Frontend-Anwendungen gegen die jeweiligen BFFs (siehe 3.5), damit UI-Erweiterungen ebenfalls unabhängig deploybar sind.

Rendering-Strategie: Client-Side-Rendering (entschieden)

- Für die authentifizierte Kernanwendung (User-/Admin-/Reviewer-/Process-Designer-UI) wird **clientseitiges Rendering (CSR)** eingesetzt, React/Next.js also primär als SPA-Framework statt mit durchgängigem Server-Side-Rendering - **Entscheidung: bewusst zugunsten geringerer Betriebs-/Ressourcenlast getroffen**, nicht mehr nur Empfehlung:
 - **SEO spielt keine Rolle:** Die gesamte Anwendung liegt hinter Authentifizierung (4.4) - der klassische SSR-Vorteil (bessere Suchmaschinenindexierung, schnellerer First Contentful Paint für anonyme Besucher) entfällt für den weit überwiegenden Teil der Anwendung.

- **Vermeidet eine zusätzliche Node-Laufzeitschicht im Backend:** Da die Backend-Services in Python laufen (1a), würde durchgängiges SSR bedeuten, dass ein separater Next.js-Node-Prozess bei jedem Seitenaufruf serverseitig Daten aus den Python-BFFs/APIs nachlädt und daraus HTML rendert - ein zusätzlicher Betriebsbestandteil (eigene Skalierung, eigenes Monitoring, eigene Fehlerquelle) ohne klaren Gegenwert, wenn die Inhalte ohnehin personalisiert/berechtigungsabhängig sind und nicht cachebar wären. Der Verzicht auf diese Schicht spart genau diesen Overhead ein.
- Die CSR-Anwendung spricht die Python-BFFs/API-Gateway (3.5) direkt als JSON-API an; Next.js wird dabei im Wesentlichen als React-Build-Tooling und Routing-Framework genutzt (ggf. mit statischem Export), nicht für dynamisches serverseitiges Rendering pro Request.
- **Gezielte Ausnahmen, wo SSR/SSG weiterhin sinnvoll bleibt:** öffentlich erreichbare, nicht-personalisierte Seiten außerhalb der eigentlichen Anwendung (Login-Seite, ggf. eine öffentliche Statusseite, Dokumentation/Hilfeseiten) - dort ist statisches oder serverseitiges Rendering unproblematisch, da kein Python-Datenzugriff pro Request nötig ist.

9. Lizenzsystem

9.1 Lizenzdimensionen

Eine Lizenz kann mehrere, unabhängig konfigurierbare Dimensionen umfassen:

Dimension	Ausprägung
Nutzeranzahl	wahlweise gleichzeitige Nutzer (concurrent sessions) oder maximale Nutzerzahl (benannte Accounts) - pro Lizenz festgelegt, welches Modell gilt
Applikationskomponenten	einzelne Services/Module sind separat lizenzierbar (z. B. CMIS-Connector, Migration-Service, Workflow-Automatisierung)
Speichervolumen	Obergrenze in GB/TB, optional unbegrenzt
Dokumentenzahl	Obergrenze an Dokumenten, optional unbegrenzt

Jede Dimension kann individuell auf "unlimited" gesetzt werden - eine Lizenz mit unbegrenzten Nutzern und Dokumenten muss ebenso abbildbar sein wie eine eng limitierte.

9.2 License Service

- Eigenständiger Service, der Lizenzschlüssel/-zertifikate verwaltet und validiert (z. B. signierte Lizenzdatei, die alle Dimensionen und Gültigkeitszeitraum enthält).
- Prüft laufend (nicht nur beim Start) aktuelle Nutzung gegen Limits, z. B. über Events aus dem Audit-/Nutzungsstrom (aktuelle gleichzeitige Sessions vom Auth Service, Dokumentenzahl vom Document Service, Speicherverbrauch aggregiert).
- Publiziert Statusänderungen (Grenze erreicht, Lizenz läuft in X Tagen ab, Lizenz ungültig) als Events - konsumiert u. a. vom Registry Service (siehe 3.2b) und vom Notification Service (Admin-Benachrichtigung).

9.3 Verhalten bei fehlender/ungültiger Lizenz

- Registry Service liefert jedem sich registrierenden Service den Lizenzstatus seiner Komponente mit.
- Ohne gültige Lizenz für eine Komponente: **Demo-Modus** (funktional eingeschränkt, z. B. nur Lesezugriff, sichtbare Wasserzeichen/Hinweise, zeitlich befristet) oder vollständige Sperre - konfigurierbar je Komponente, welches Verhalten greift.
- Überschreitung eines Nutzungslimits (z. B. Dokumentenzahl) blockiert nicht rückwirkend bestehende Daten, verhindert aber neue Anlagen, bis das Limit wieder eingehalten oder die Lizenz erweitert wird.
- Der Lizenzstatus ist auch in der Admin-UI jederzeit einsehbar (siehe 7), inkl. Restlaufzeit und Auslastung je Dimension.

10. Betrieb & Erweiterbarkeit

10.1 Monitoring & Observability

Monitoring wird als eigenständiger, konfigurierbarer Baustein behandelt - nicht als fixes "an oder aus", sondern granular steuerbar bis auf Ebene einzelner Messpunkte.

Sensor-Konzept

- Jeder Service bringt eine Menge an **Sensoren** mit: einzelne, klar benannte Messpunkte (z. B. `document.upload.duration`, `permission.check.latency`, `storage.backend.s3-primary.write.errors`, `registry.service.heartbeat.miss`, `license.usage.documents.current`).
- Sensoren sind fachlich gruppiert (z. B. nach Service, nach Kategorie wie Performance/Fehler/Kapazität/Sicherheit), damit sie sich in der Admin-UI übersichtlich aktivieren/deaktivieren lassen, ohne jeden einzelnen Sensor manuell durchgehen zu müssen.
- Jeder Sensor kennt seine eigenen Kosten (CPU/IO-Overhead, Kardinalität der Labels) - hochauflösende oder teure Sensoren sind klar gekennzeichnet, damit ein "alles an" nicht versehentlich das System spürbar belastet.
- **Grundsatz "maximale Abdeckung, aber nur bei Bedarf"**: Grundsätzlich soll alles messbar sein, was sich sinnvoll messen lässt - es gibt keine künstliche Beschränkung des Sensor-Katalogs. Ressourcenschonung entsteht nicht durch weniger verfügbare Sensoren, sondern dadurch, dass ein Sensor **nur dann überhaupt Daten erzeugt**, wenn er im Monitoring Service aktiv geschaltet ist: Ist ein Sensor deaktiviert, unterbleibt bereits die Erfassung im jeweiligen Service (kein Erzeugen, kein Zwischenspeichern, kein Zurückspielen an den Metrics-Endpoint) - nicht nur das Anzeigen/Exportieren der Daten. Wer das Monitoring bewusst schlank hält, spart also tatsächlich Systemressourcen ein, statt nur die Sichtbarkeit zu reduzieren.

Konfigurierbarkeit (default-an/default-aus + Feinsteuerung)

- Globale Grundeinstellung pro Installation: **"alles überwachen"** (alle verfügbaren Sensoren aktiv) oder **"nichts überwachen"** (Monitoring komplett deaktiviert) als Ausgangspunkt.
- Darauf aufbauend gezielte Overrides: einzelne Sensoren oder ganze Sensor-Gruppen unabhängig von der Grundeinstellung ein-/ausschalten (z. B. Grundeinstellung "nichts", aber gezielt nur die Sensoren des Storage Service für ein aktuelles Kapazitätsproblem aktivieren; oder Grundeinstellung "alles", aber teure Tracing-Sensoren im Dauerbetrieb bewusst abschalten).
- Konfiguration ist - wie andere Systemeinstellungen auch - Teil des JSON-Konfigurationsexports (7.3) und pro Installation individuell einstellbar; Änderungen wirken sich ohne Neustart der betroffenen Services aus (Sensoren werden zur Laufzeit an-/abgeschaltet).
- Die Sensor-Konfiguration selbst ist auditiert (wer hat wann welche Sensoren aktiviert/

deaktiviert) - relevant, da das Deaktivieren von Sicherheits-relevanten Sensoren selbst ein sicherheitsrelevanter Vorgang ist.

Architektur & Anbindung an Standardlösungen

- Jeder Service exponiert seine aktiven Sensoren über einen einheitlichen **Metrics-Endpoint** im Prometheus-Exposition-Format (De-facto-Standard, direkt scrapebar).
- Der Registry Service führt zusätzlich zur Service-Discovery (3.2) eine **Sensor-Registry**: welche Services bieten welche Sensoren an, welche sind aktuell aktiv - das erlaubt es, Prometheus-Scrape-Konfiguration oder CheckMK-Discovery weitgehend automatisch aus der Registry abzuleiten, statt sie manuell pro neuem Service zu pflegen (passt zum "Dazustellen"-Prinzip: ein neuer Service bringt seine Sensoren mit und ist sofort überwachbar).
- **Prometheus**: primärer Scrape-Konsument der Metrics-Endpoints.
- **Grafana**: Konsument von Prometheus (bzw. optional direkt unterstützter Datenquellen) für Dashboards - das System selbst liefert keine eigene Grafana-Ersatz-UI, sondern stellt sinnvolle Default-Dashboard-Definitionen als exportierbare JSON-Vorlagen bereit (Startpunkt, keine Pflichtnutzung).
- **CheckMK**: Anbindung über dessen Standard-Prometheus-Special-Agent oder alternativ eigenen CheckMK-kompatiblen Check-Plugin, damit klassische IT-Monitoring-Landschaften (Alarmierung, Eskalation, SLA-Reporting) das System einbinden können, ohne eine Parallelwelt zu Prometheus/Grafana pflegen zu müssen.
- Das System schreibt sich damit nicht auf ein bestimmtes Monitoring-Tool fest, sondern liefert standardkonforme Metriken, die von gängigen Lösungen konsumiert werden - eigene Anbindungen weiterer Tools sind über denselben Metrics-Endpoint grundsätzlich möglich, ohne Änderungen am DMS selbst.

Abgrenzung zu Audit/Tracing

- Monitoring (Sensoren/Metriken) ist bewusst getrennt vom Audit-Trail (5.3): Metriken sind aggregiert, anonym/technisch und primär für Betrieb/Kapazitätsplanung/Alarmierung gedacht, während der Audit-Trail personenbezogen, vollständig und unveränderlich für Nachvollziehbarkeit/Forensik (5.4) ist. Beide Ströme können denselben Event-Bus (3.4) als technische Grundlage nutzen, bedienen aber unterschiedliche Zwecke und unterliegen unterschiedlichen Zugriffs-/Aufbewahrungsregeln.
- Verteiltes Tracing (z. B. OpenTelemetry) über Service-Grenzen hinweg ergänzt die Sensor-Metriken um den Verlauf einzelner Anfragen (Gateway → Document Service → Permission

Service → Constraint Engine → Audit Service) - ebenfalls sensorartig konfigurierbar (z. B. nur für einen Teil der Anfragen aktiv, um Overhead zu begrenzen: Sampling-Rate als eigener Konfigurationsparameter).

- Zentrale Log-Aggregation (z. B. via Event-Bus oder Log-Shipping) ergänzt Metriken und Traces um Volltext-Fehlerdetails.

10.2 Deployment & Skalierung

- Jeder Service unabhängig containerisiert und horizontal skalierbar (mehrere Instanzen je Servicetyp, Registry verteilt Last/Erreichbarkeit).
- Neue Servicetypen (weitere Connectoren, neue Regeltypen, neue UI-Module) werden durch Deployment einer neuen Instanz "dazugestellt" und über die Registry automatisch eingebunden - ohne Downtime oder Eingriff in bestehende Services.
- Die Platzierung und laufende Umverteilung dieser Instanzen übernimmt der Plugin Orchestration Service (3.8) automatisch und zeitprofil-bewusst, statt dass Betreiber manuell festlegen müssen, welcher Dienst auf welchem Server läuft.
- **Abgrenzung zu klassischen EIM-Plattformen:** Systeme wie nscale sind historisch nicht durchgängig auf zustandslose horizontale Skalierung jeder einzelnen Komponente ausgelegt - einzelne Kernkomponenten lassen sich dort nicht beliebig vervielfachen. Das hier beschriebene Konzept vermeidet das strukturell, indem jeder Service von Grund auf zustandslos entworfen ist (Zustand liegt in Shared DB/Event-Bus/Storage, nicht im Service-Prozess) und beliebig viele Instanzen parallel laufen können.

10.3 Sicherheitsaspekte (ergänzend)

- Virensan verpflichtend vor Freigabe eines Uploads - nicht bestandene Uploads landen im eigenen Quarantäne-Bereich (2.5) statt verworfen oder stillschweigend abgelehnt zu werden.
- Verschlüsselung ruhender Daten (at rest) und Transportverschlüsselung (TLS) zwischen allen Services.
- Getrennte DB-Credentials pro Service (siehe 3.1) auch als Sicherheitsmaßnahme, nicht nur Architekturprinzip.

10.4 Backup & Restore

Backup/Restore ist als eigenständiger, von Anfang an mitgedachter Bestandteil des Konzepts

vorgesehen - kein nachträglich aufgesetztes Betriebsthema, gerade weil die verteilte Architektur (mehrere Speicherorte, Shared DB mit getrennten Schemata, Event-Bus) eine unkoordinierte Sicherung einzelner Komponenten besonders fehleranfällig machen würde.

Was gesichert wird

- **Shared Database (3.1):** vollständige, konsistente Sicherung aller Schemata gemeinsam (nicht schemaweise getrennt zu unterschiedlichen Zeitpunkten, sonst verlieren die Owner-Schema-Beziehungen ihre Konsistenz) - klassisch über `pg_basebackup` /kontinuierliche WAL-Archivierung für Point-in-Time-Recovery, nicht nur periodische Voll-Dumps.
- **Storage-Backends (3.6):** Sicherung ergänzend zur bereits bestehenden Redundanz (mehrere gleichzeitig beschriebene Backends) - Redundanz schützt vor Backend-Ausfall, aber nicht vor logischen Fehlern (versehentliches/böswilliges Löschen, Bug in einem Service), die sich sonst auf alle redundanten Kopien gleichermaßen auswirken würden. Deshalb zusätzlich ein echtes, zeitversetztes Backup (z. B. Objektversionierung mit Aufbewahrung gelöschter/überschriebener Versionen, oder ein separates Backup-Ziel außerhalb des regulären Storage-Zielsatzes).
- **Event-Bus (3.4):** je nach gewählter Technologie (NATS JetStream/Kafka) eigene Snapshot-/Retention-Mechanismen; da Audit-relevante Events ohnehin dauerhaft im Audit-Service-Schema landen (5.3), ist der Event-Bus selbst primär für kurzfristige Wiederherstellung/Replay relevant, nicht als alleinige Quelle der Wahrheit.
- **Konfiguration (7.3-Export)** als zusätzlicher, unabhängiger Wiederherstellungspunkt - erlaubt im Extremfall auch einen "Neuaufbau" der Systemkonfiguration unabhängig von einem Datenbank-Restore.
- Keycloak (4.4) nutzt dieselbe Postgres-Instanz (3.1) und ist damit in der regulären DB-Sicherung automatisch mit erfasst - kein separater Mechanismus nötig.

Reihenfolge bei der Wiederherstellung

Die Reihenfolge ist bewusst festgelegt, um Inkonsistenzen zwischen Komponenten zu vermeiden (z. B. Datenbankeinträge, die auf im Storage noch nicht vorhandene Objekte verweisen):

1. **Konsistenzanker festlegen:** Storage-Backup und Datenbank-Backup werden auf einen gemeinsamen, koordinierten Zeitpunkt festgelegt (Point-in-Time-Konsistenz) - ein Restore auf zeitlich unterschiedliche Stände von Storage und DB ist die Hauptursache für Inkonsistenzen und wird als Vorgehen ausgeschlossen.
2. **Storage-Backends zuerst wiederherstellen** - die referenzierten Objekte müssen

existieren, bevor die Datenbank (die auf sie verweist) wieder aktiv genutzt wird, sonst entsteht ein Zeitfenster mit "Karteileichen"-Referenzen.

3. **Datenbank auf denselben Konsistenzzeitpunkt wiederherstellen** (Point-in-Time-Recovery über die WAL-Archivierung).
4. **Löschabgleich durchführen** (siehe unten) - zwingender Schritt vor der Wiederfreigabe des Systems, kein optionaler Nachlauf.
5. **Event-Bus** ggf. auf denselben Zeitpunkt zurücksetzen bzw. neu aufbauen (bei den meisten Konsumenten unkritisch, da sie ohnehin aus der wiederhergestellten DB rekonstruierbare Zustände halten - z. B. der Search-Index, siehe Punkt 7).
6. **Registry, License Service und übrige Services regulär neu starten** - sie registrieren sich wie im Normalbetrieb (3.2) neu, keine Sonderbehandlung nötig, da der Registry-Zustand ohnehin als "aktuell erreichbar" laufend neu ermittelt wird, nicht aus einem Backup wiederhergestellt werden muss.
7. **Abgeleitete/rekonstruierbare Zustände zuletzt neu aufbauen:** Der Search-Index (Search Service) muss nicht aus einem eigenen Backup wiederhergestellt werden, sondern kann - sofern zeitlich vertretbar - aus dem wiederhergestellten Datenbestand neu indiziert werden; reduziert die Zahl der Komponenten, die überhaupt ein eigenes, konsistentes Backup benötigen.

Verpflichtender Löschabgleich nach Restore (Zusammenspiel mit 5.2a)

Der wichtigste Sonderfall: Ein Restore auf einen Zeitpunkt *vor* einer inzwischen durchgeführten Zwangslöschung (5.2a) würde das eigentlich rechtlich zu löschende Dokument unbeabsichtigt wiederherstellen ("Auferstehung" gelöschter Daten) - das muss das System aktiv verhindern, nicht dem Betriebspersonal überlassen.

- **Löschregister als eigenständiger, vom Restore unberührter Nachweis:** Zusätzlich zum reduzierten regulären Audit-Log wird jede physische Zwangslöschung (5.2a) in einem eigenen, separat und häufiger gesicherten **Löschregister** vermerkt (Objekt-ID, Löschezitpunkt, Löschgrund) - dieses Register wird bewusst **nicht** auf denselben Zeitpunkt zurückgesetzt wie der Rest der Datenbank bei einem Restore, sondern bleibt auf dem aktuellsten Stand erhalten (z. B. durch separate, kontinuierliche Sicherung/Replikation außerhalb des eigentlichen Restore-Vorgangs).
- **Automatischer Abgleich als Pflichtschritt:** Nach jedem Restore prüft das System automatisiert, ob unter den wiederhergestellten Objekten welche sind, die laut Löschregister **nach dem Backup-Zeitpunkt, aber vor dem Restore-Zeitpunkt** bereits zwangsgelöscht worden waren. Für jeden gefundenen Fall wird die physische Löschung

automatisch erneut ausgeführt - derselbe Mechanismus wie bei der ursprünglichen Zwangslöschung (5.2a), inklusive Audit-Eintrag, dass dies im Rahmen eines Restore-Abgleichs erfolgte.

- Erst nach erfolgreichem Abschluss dieses Abgleichs gilt der Restore als vollständig und das System wird für den regulären Betrieb wieder freigegeben - nicht vorher, um zu verhindern, dass in der Zwischenzeit jemand auf ein Dokument zugreift, das eigentlich bereits hätte gelöscht sein müssen.
- Dieselbe Logik gilt sinngemäß auch für reguläre (nicht-physische) Löschungen und Aussonderungen (5.6), auch wenn dort die Konsequenz einer versehentlichen "Wiederherstellung" weniger schwer wiegt als bei einer rechtlich verpflichtenden Zwangslöschung.

Durchführung

- Backup/Restore-Vorgänge selbst sind auditert (5.3) und lösen bei Restore automatisch einen Wartungsmodus aus, vergleichbar dem Not-Shutdown-Wartungszustand (4.8) - regulärer Nutzerzugriff bleibt bis zum Abschluss des Löschatgleichs gesperrt.
- Wiederherstellung ist testbar/übbar vorzusehen (regelmäßige Restore-Tests auf eine isolierte Testumgebung), um sicherzustellen, dass die Reihenfolge und der Löschatgleich im Ernstfall tatsächlich funktionieren - eine Sicherung, die nie erfolgreich zurückgespielt wurde, ist keine verlässliche Sicherung.

10.5 Unterbrechungsfreie Updates (Rolling Updates)

Updates sollen die Nutzung der Anwendung nach Möglichkeit nicht unterbrechen - das ist keine Zusatzfunktion, sondern eine direkte Konsequenz aus der konsequent zustandslosen Service-Architektur (Leitprinzip 1/6): Da jeder Service seinen Zustand ausschließlich in Shared DB, Event-Bus und Storage hält (nie im eigenen Prozessspeicher), lässt sich eine einzelne Service-Instanz grundsätzlich ersetzen, ohne dass fachlicher Zustand "mitgenommen" werden müsste.

Grundmechanismus: Parallelbetrieb statt In-Place-Austausch

- Beim Update einer Komponente wird zunächst eine **neue Instanz in der neuen Version zusätzlich zur laufenden alten Instanz** gestartet, nicht die alte direkt ersetzt.
- Die neue Instanz registriert sich beim Registry Service (3.2) und durchläuft dort einen Health-/Readiness-Check, bevor sie überhaupt als einsatzbereit gilt.

- Sobald die neue Instanz als bereit erkannt ist, beginnt die Registry, **neue** Anfragen bevorzugt/ausschließlich an die neue Instanz zu routen, während die alte Instanz in einen **Draining-Status** versetzt wird - denselben Zustand, den der Plugin Orchestration Service (3.8) bereits für die schonende Umverteilung bei Lastspitzen nutzt. Es handelt sich technisch um denselben Drain-Mechanismus, hier nur zu einem anderen Anlass (Update statt Rebalancing) eingesetzt - kein zweites, parallel gepflegtes Konzept.
- Die alte Instanz nimmt im Draining-Status keine neuen Aufgaben mehr an, schließt aber bereits laufende Vorgänge regulär ab (z. B. eine laufende BPMN-Prozessausführung, ein laufender Storage-Schreibvorgang, ein offener Signature-Task, 3.10) - erst nach vollständigem Abschluss aller offenen Vorgänge wird die alte Instanz terminiert.
- Für den Fall, dass sich die neue Version nach der Umschaltung als fehlerhaft erweist, bleibt die alte Instanz bis zum Abschluss des Drains ansprechbar - ein schnelles Zurückschalten (Rollback) ist damit möglich, solange der Drain der alten Instanz noch nicht vollständig abgeschlossen ist.

Grenzen: Persistenzebene als Sonderfall

- Dieses Verfahren funktioniert vollständig unterbrechungsfrei für Updates, die **keine Änderung an Datenbankschema oder Storage-Format** mit sich bringen (reine Code-/Logikänderungen an einem oder mehreren Services) - das ist der Normalfall bei kleineren Patches/Updates und wird als Standardverfahren erwartet.
- Für Updates, die eine **Schemaänderung der Shared DB (3.1)** erfordern, gilt eine differenziertere Betrachtung:
 - Wo möglich, wird nach dem etablierten **Expand/Contract-Muster** (auch "Parallel Change" genannt) vorgegangen: Ein erster Migrationsschritt ergänzt neue Strukturen (Spalten/Tabellen), ohne bestehende zu entfernen ("Expand") - alte und neue Serviceversionen können die Datenbank währenddessen parallel nutzen, da die alten Strukturen weiterhin existieren. Erst nachdem **alle** Instanzen auf die neue Version aktualisiert und die alten vollständig gedraint sind, entfernt ein zweiter, nachgelagerter Migrationsschritt die nicht mehr benötigten alten Strukturen ("Contract"). Damit bleiben auch DB-Schema-Änderungen in aller Regel unterbrechungsfrei durchführbar, erfordern aber zwei getrennte Rollout-Schritte statt eines einzigen.
 - Für Änderungen am **Storage-Format** (3.6) gilt sinngemäß dasselbe Prinzip: neue und alte Objektformate bestehen während der Übergangsphase parallel, bis alle Konsumenten auf die neue Version migriert sind.

- Es wird bewusst nicht der Anspruch erhoben, dass **jede denkbare** strukturelle Änderung unterbrechungsfrei möglich ist - bei einer grundlegenden, nicht sinnvoll in Expand/Contract zerlegbaren Restrukturierung kann ein kurzes, geplantes Wartungsfenster im Einzelfall unumgänglich sein. Das ist dann aber eine bewusst geplante, klar kommunizierte Ausnahme (mit Vorankündigung, analog zum Wartungsmodus bei Restore, 10.4), nicht der Regelfall.
- API-Kompatibilität zwischen Services während der Übergangsphase ist Voraussetzung: Da alte und neue Serviceversionen für die Dauer des Drains gleichzeitig mit anderen, noch nicht aktualisierten Services kommunizieren, muss jede Serviceversion innerhalb eines Update-Rollouts abwärts-/vorwärtskompatibel zur jeweils anderen bleiben (keine brechenden API-Änderungen innerhalb eines einzelnen Rollout-Vorgangs) - dieselbe Grunddisziplin, die bereits für die Versionskompatibilität zwischen föderierten Installationen (7.4) beschrieben ist, hier auf Service-Ebene innerhalb einer einzelnen Installation angewendet.

11. Architekturübersicht (Diagramm)

Grobe Darstellung der wichtigsten Services und ihrer Kommunikationswege. Bewusst vereinfacht (nicht jeder Service-zu-Service-Aufruf ist eingezeichnet) - Fokus liegt auf den strukturellen Beziehungen: Clients → Gateway → Fachservices, Control Flow über die Registry, Data Flow über Event-Bus/Shared-DB/Storage.

```

flowchart TB
    subgraph Clients
        UI_User["User Web-UI"]
        UI_Admin["Admin Web-UI"]
        UI_Review["Reviewer/Approval-UI"]
        UI_Migration["Migrations-Konsole"]
        CLI["CLI-Tool"]
    end
    GW["API-Gateway / BFF-Schicht"]
    subgraph Control ["Control Flow"]
        REG["Registry"]
        Service_Discovery["Service<br/>(Discovery + Lizenz + Sensoren)"]
        ORCH["Plugin Orchestration"]
        Service_Placement["Service<br/>(Platzierung, Lastverteilung)"]
        LIC["License Service"]
        FLEET["Fleet-Management<br/>(mehrere Installationen, optional)"]
    end
    subgraph Core ["Kern-Fachservices"]
        AUTH["Auth Service<br/>(AD + Lokal)"]
        PERM["Permission Service<br/>(RBAC, Vererbung, 4-Augen)"]
        DOC["Document Service<br/>(CRUD, Version, Lock)"]
        FOLD["Folder Service"]
        OBJ["Object-Type Service"]
        RULE["Rule/Constraint Engine"]
        WF["BPMN Process Engine"]
        CASE["Case Service (Umlaufmappen)"]
    end
    subgraph Support ["Unterstützende Services"]
        AUDIT["Audit Service"]
        REPORT["Reporting Service<br/>(Standardberichte + Forensik-Trace)"]
        SEARCH["Search Service"]
        RENDER["Rendering/Preview Service"]
        AV["Virus-Scan Service"]
    end
    Clients --> GW
    GW --> Control
    Control --> Core
    Core --> Support
  
```

OCR["OCR Service"] SIG["Signature Service"] NOTIFY["Notification Service"] MIG["Migration/
Transfer Service"] CFG["Config Im-/Export Service"] QUERY["Query/Trace Service"] end
 subgraph Federation["Föderation (außerhalb der Installation)"] HUB["Federation Hub
Service
(Adressbuch + Schaltzentrale)"] end subgraph Connectors["Connector Services"]
 CMIS["CMIS-Connector"] WEBDAV["WebDAV-Connector"] CUSTOM["Custom-Connector(en)"]
 end subgraph StorageLayer["Storage-Abstraktion"] STOR["Storage Service"] S3A["S3 Backend
A"] S3B["S3 Backend B"] NFS["NFS Backend"] end BUS(["Event-Bus"]) DB(["Shared
DB
(getrennte Schemata)"]) subgraph Monitoring["Monitoring (extern)"]
 PROM["Prometheus"] GRAF["Grafana"] CHECKMK["CheckMK"] end UI_User --> GW UI_Admin
 --> GW UI_Review --> GW UI_Migration --> GW CLI --> GW GW --> AUTH GW --> PERM GW
 --> DOC GW --> FOLD GW --> WF GW --> SEARCH GW --> REPORT GW --> CFG GW -->
 QUERY QUERY --> AUDIT QUERY --> REPORT QUERY -.Sensoren.-> PROM WF <--
 Prozessübergabe --> HUB GW -. Erreichbarkeit erfragen .-> REG AUTH -. registriert sich .->
 REG PERM -. registriert sich .-> REG DOC -. registriert sich .-> REG WF -. registriert sich .-> REG
 STOR -. registriert sich .-> REG CMIS -. registriert sich .-> REG WEBDAV -. registriert sich .->
 REG CUSTOM -. registriert sich .-> REG REG <-- Lizenzstatus --> LIC LIC <-. mehrere
 Installationen .-> FLEET ORCH <-- Platzierungsdaten --> REG ORCH -. beobachtet Sensoren .->
 PROM DOC --> RULE FOLD --> RULE DOC --> PERM FOLD --> PERM WF --> PERM WF -->
 RULE WF --> CASE CASE --> DOC CASE --> DB DOC --> OBJ FOLD --> OBJ DOC --> STOR
 STOR --> S3A STOR --> S3B STOR --> NFS DOC --> AV DOC --> OCR OCR --> SEARCH WF --
 > SIG SIG --> DOC DOC --> RENDER CMIS --> DOC WEBDAV --> DOC CUSTOM --> DOC MIG
 --> DOC MIG --> FOLD MIG --> PERM AUTH --> DB PERM --> DB DOC --> DB FOLD --> DB
 OBJ --> DB WF --> DB AUDIT --> DB LIC --> DB AUTH ==> BUS PERM ==> BUS DOC ==>
 BUS FOLD ==> BUS WF ==> BUS MIG ==> BUS LIC ==> BUS BUS ==> AUDIT BUS ==>
 SEARCH BUS ==> NOTIFY BUS ==> REPORT AUDIT --> REPORT CFG --> OBJ CFG --> RULE
 CFG --> WF CFG --> PERM Core -.Sensoren.-> PROM Support -.Sensoren.-> PROM
 StorageLayer -.Sensoren.-> PROM PROM --> GRAF PROM --> CHECKMK

Legende (informell):

- Durchgezogene Pfeile: direkter, synchroner API-Aufruf (Data Flow).
- Gepunktete Pfeile: Registrierung/Abfrage über den Registry Service (Control Flow) bzw. Sensor-Abgriff.
- Dicke Pfeile (**==>**): Event-Publishing auf den Event-Bus.
- Datenbank-Symbol: Zugriff ausschließlich auf das jeweils eigene Schema (siehe 3.1), kein Cross-Schema-Zugriff dargestellt oder vorgesehen.

12. Markteinordnung & Gap-Liste

Vergleich des Konzepts gegen zwei etablierte Marktlösungen: **nscale** (Ceyoniq, proprietäres EIM/ECM-System) und **VIS-Suite** (PDV, proprietäre ECM-Plattform für die öffentliche Verwaltung). Ziel: im Kernfunktionsumfang mindestens gleichwertig sein, an strukturellen Schwächen der Marktlösungen (z. B. Skalierung) bewusst besser.

Hinweis: Ein drittes, in der ursprünglichen Anfrage genanntes System ("Ldoc") konnte nicht eindeutig als etabliertes Marktprodukt identifiziert werden und wurde daher hier nicht einbezogen.

12.1 Vergleichstabelle

Bereich	Dieses Konzept	nscale (Ceyoniq)	VIS-Suite (PDV)
Architektur	Cloud-native Microservices, Shared DB mit getrennten Schemata, Event-Bus, Plugin-Orchestrierung	Modulare EIM- Plattform, klassisch on-prem/Cloud- hybrid, nicht durchgängig zustandslos- horizontal skalierbar	Modulkonzept (Poststelle, Archiv, ...), traditionell gewachsene ECM- Architektur
Horizontale Skalierung	Jeder Service zustandslos, beliebig instanzierbar, automatische zeitprofil-bewusste Lastverteilung über Plugin Orchestration Service	Eingeschränkt - Kernkomponenten nicht beliebig vervielfachbar	Nicht öffentlich dokumentiert, keine als Kernfeature beworbene horizontale Skalierung
Unterbrechungsfreie Updates	Explizit vorgesehen (Parallelbetrieb + Drain-Mechanismus,	Nicht öffentlich als Kernfeature dokumentiert	Nicht öffentlich als Kernfeature dokumentiert

Bereich	Dieses Konzept	nscale (Ceyoniq)	VIS-Suite (PDV)
	Expand/Contract bei Schemaänderungen, siehe 10.5) - direkte Folge der zustandslosen Architektur		
Auditierbarkeit/ Revisionssicherheit	Strukturell über Event-Bus erzwungen, Hash-Chain-Idee, TR-ESOR als Referenz	Revisionssichere Archivierung vorhanden, für TR-ESOR-konforme Langzeitarchivierung erweiterbar	LegalHold, Zugriffs-/Veränderungsschutz vorgesehen, Langzeitspeicherung über externe Partner
Workflow/ Prozessautomatisierung	BPMN-2.0-basiert (Modellierung + Import), Constraint-gekoppelt, konfigurierbares Vier-Augen-Prinzip, Multi-Instanz-/föderierte Prozessschritte	No-Code-Prozessplattform (PAP) für Fachabteilungen	Prozessautomation Kernbestandteil der aktuellen Produktgeneration
Berechtigungen	RBAC mit Vererbung, ordnerweise, Vier-Augen pro Aktionstyp konfigurierbar	Vorhanden, Detailtiefe öffentlich nicht dokumentiert	Zugriffsmandanten-Konzept, im Public-Sector-Kontext etabliert
KI-Funktionen	Nicht vorgesehen (Kandidat für spätere Erweiterung, siehe 12.2)	KI-Chat, automatische Zusammenfassungen, KI-gestützte Prozessunterstützung angekündigt	KI-Integration Teil der aktuellen Produktgeneration
Mobile Clients	Nicht vorgesehen (nur	Web-/Desktop-	Eigener MobileClient

Bereich	Dieses Konzept	nscale (Ceyoniq)	VIS-Suite (PDV)
	Web-UIs, siehe 12.2)	fokussiert	für ortsunabhängigen Aktenzugriff
E-Signatur	Explizit vorgesehen (SES/AES/QES nach eIDAS, Broker vor externen QTSPs, PAdES/XAdES/CAAdES, siehe 3.10)	Standardfunktion	Über Standardprozesse im E-Akte-Umfeld abgedeckt
OCR/Texterkennung	Explizit vorgesehen, fester Bestandteil der Eingangserfassung, plugin-basierte Engine-Wahl (siehe 3.9)	Kernbestandteil der Eingangserfassung	Klassische Scan-/Erfassungsfunktionen etabliert
ERP/Fachverfahren-Integration	Nur generische Connector-Architektur (CMIS/WebDAV/Custom), keine vorgefertigten Standard-Konnektoren	Bidirektionale DATEV-Schnittstelle, SAP-S/4HANA-zertifiziert	Integration in Fachverfahren/E-Government-Basisinfrastrukturen als Standardanforderung
Storage-Redundanz (Multi-Backend gleichzeitig)	Explizit vorgesehen, granular konfigurierbar - Alleinstellungsmerkmal	Nicht öffentlich in dieser Konfigurierbarkeit dokumentiert	Nicht öffentlich in dieser Konfigurierbarkeit dokumentiert
Konfigurierbares Monitoring (Sensor-Ebene, Standard-Exporter)	Explizit vorgesehen (Prometheus/Grafana/CheckMK)	Zentrale Systemüberwachung vorhanden, kein dokumentierter Standard-Exporter-Ansatz	Nicht öffentlich dokumentiert

Bereich	Dieses Konzept	nscale (Ceyoniq)	VIS-Suite (PDV)
Custom Objekttypen + freie Constraints	Explizit vorgesehen (frei modellierbar)	Kategorisierung nach eigenen Kriterien, Constraint-Tiefe nicht dokumentiert	Eher vorgegebene Aktenplan-/Schriftgutobjekt-Struktur
Umlaufmappen & Vorgangsakten	Explizit vorgesehen (Referenzstruktur, archivierbar, prozessspezifische Bearbeitungskopien, siehe 2.3)	Aktenbildung im E-Akte-Umfeld etabliert	Kernkonzept der E-Akte im Public-Sector-Umfeld
Poststelle/ Sonderbereiche (Posteingang/-ausgang, Papierkorb inkl. Verschlussachen-Variante, Quarantäne, Kontakte, Aussonderungs-Zugriff, Vorlagen)	Explizit vorgesehen als eigene, vom regulären Ordnerbaum getrennte Bereiche mit eigenem Zugriffsregime (siehe 2.5)	Poststellen-/Eingangsmodule etabliert	Kernbestandteil des klassischen Schriftgutverwaltungs-Modulkonzepts
Barrierefreiheit/ Ausfallsicherheit (Ersatzdarstellungen)	Explizit vorgesehen (automatisch erzeugte Alternativformate, siehe 2.4)	Nicht öffentlich als eigenständiges Feature dokumentiert	Nicht öffentlich als eigenständiges Feature dokumentiert
Mehrfachinstallationen (Reseller-Szenario)	Explizit vorgesehen (isolierte Installationen + Fleet-Management)	Nicht als Kernfeature beworben	Nicht als Kernfeature beworben
Konfigurations-Delta zwischen Installationen	Explizit vorgesehen (Basisinstanz-Vergleich, konfigurierbarer Umfang, Ignore-Regex für	Nicht öffentlich als eigenständiges Feature dokumentiert	Nicht öffentlich als eigenständiges Feature dokumentiert

Bereich	Dieses Konzept	nscale (Ceyoniq)	VIS-Suite (PDV)
	Namensabweichungen, siehe 7.5)		
Lizenzmodell	Multi-dimensional, technisch über Registry/License Service durchgesetzt, "unlimited" je Dimension möglich	Marktüblich modulbasiert, bei Teilprodukten nach Nutzungsvolumen gestaffelt	Modul-/nutzerbasiert, öffentliche Detailtiefe gering
Open Source	Ja (geplant)	Nein, proprietär	Nein, proprietär

12.2 Kandidaten für spätere Erweiterung (bewusst nicht Teil des aktuellen Konzepts)

Diese Punkte sind identifizierte Lücken gegenüber dem Marktumfeld, aber kein Bestandteil dieser Konzeptversion - die Architektur (Plugin-/Connector-Prinzip, 3.3/3.8) ist so ausgelegt, dass sie sich später ohne Eingriff ins Grundgerüst ergänzen lassen. **E-Signatur und OCR wurden zwischenzeitlich aus dieser Liste in den Kernumfang übernommen (siehe 3.9/3.10)** - verbleibende Kandidaten in Reihenfolge nach festgelegter Priorität (absteigend):

1. **Vorgefertigte Standard-Konnektoren zu gängiger ERP-/Fachverfahrenssoftware** (z. B. DATEV, SAP) - über die generische Connector-Architektur technisch möglich, aber nicht vorkonfiguriert.
 2. **Native Mobile Clients** (bislang nur Web-UIs vorgesehen).
 3. **KI-Funktionen** (z. B. Dokumenten-Chat, automatische Zusammenfassung, Prozessunterstützung) - als optionaler, dazustellbarer Service denkbar; niedrigste Priorität der verbleibenden Kandidaten, konsistent mit der Entscheidung in 5.4, KI-gestützte Auswertung aktuell out of scope zu halten.
-

13. Offene Punkte für die nächste Konkretisierungsrunde

- **Technologieauswahl - Stand der Entscheidungen:** DB **Postgres** (entschieden), Frontend-Framework **React/Next.js**, überwiegend **clientseitiges Rendering (CSR)** zur **Vermeidung einer zusätzlichen Node-SSR-Laufzeitschicht** (entschieden, siehe 8), Backend-Sprache **Python/FastAPI** als **Startpunkt, dank durchgängiger API-Kopplung service-weise durch andere Sprachen ersetzbar** (entschieden, siehe 1a, inkl. konkreter Bibliotheksempfehlungen für Storage-/Event-Bus-/Auth-Anbindung), Event-Bus **NATS JetStream** als **Standard, Kafka als Option für große Installationen** (empfohlen, siehe 3.4), Auth-Protokoll **OIDC auf Basis von Keycloak** (empfohlen, siehe 4.4), Query-Konsolen-Parser **pghost/libpg_query** (entschieden, siehe 1a/6.1), BPMN-Engine **SpiffWorkflow** mit `bpmn-js-spiffworkflow` als Editor-Basis (empfohlen, siehe 1a/7.1).
- Reporting: genauer Katalog der Standardberichte (fachliche Priorisierung durch Stakeholder nötig).
- Zwangslöschung nach Frist (5.2a): Werkseinstellung, ob der Löschgrund optional oder verpflichtend ist, sollte final festgelegt werden (aktuell konfigurierbar gelassen, keine Systemvorgabe) - fachliche Entscheidung, gern zusammen mit den ersten Zielkunden/Objekttypen. Ebenso offen: konkrete Standardvorlaufzeit für die optionale Löscherinnerung, falls dafür ein Systemdefault sinnvoll ist statt komplett freier Konfiguration.
- Backup & Restore (10.4): konkrete Backup-Technologie je Storage-Backend-Typ (z. B. Objektversionierung vs. separates Backup-Ziel bei S3-kompatiblen Backends, Snapshot-Mechanismus bei NFS), genaues Sicherungsintervall für das Löschregister (kontinuierlich/nahezu Echtzeit wird empfohlen, konkrete technische Umsetzung noch offen), sowie Testintervall für die vorgesehenen regelmäßigen Restore-Tests.
- Unterbrechungsfreie Updates (10.5): konkrete Umsetzung des Parallelbetriebs/Drain-Mechanismus - eigenständige Orchestrierungslogik im Plugin Orchestration Service vs. Nutzung vorhandener Plattform-Primitive (z. B. Kubernetes/OpenShift Rolling-Update-Ressourcen direkt), sowie ob/wie eine schrittweise (Canary-)Umschaltung mit nur einem Teil des Datenverkehrs auf die neue Version vor der vollständigen Umstellung unterstützt werden soll.
- Query & Trace Konsole (6.1): konkrete Erweiterungsgrammatik der DMS-spezifischen Zusatzkonstrukte auf Basis von pghost/libpg_query (Parser-Basis ist entschieden, siehe 1a). Die konkrete Liste der als "kritisch" zu markierenden Tabellen/Objektklassen mit

zwingendem Vier-Augen-Prinzip (6.1) kann erst nach Festlegung des konkreten Datenbankschemas (Tabellenstruktur je Owner-Schema) sinnvoll erstellt werden - bewusst vertagt, kein technisches Hindernis.

- Plugin Orchestration Service (3.8): Platzierungsalgorithmus ist grundsätzlich entschieden (Plattform-Scheduler bevorzugt, First Fit Decreasing als Fallback) - offen bleibt der genaue Detailgrad des Lastprofil-Formats (welche Zeitprofil-Kategorien konkret unterschieden werden) sowie die exakte Gewichtung zwischen Zeitprofil-Gruppierung und reiner Ressourcengröße innerhalb des FFD-Verfahrens.
- BPMN-Workflow Engine (7.1): SpiffWorkflow ist LGPLv3-lizenziert - im Gegensatz zu den bislang gewählten Bausteinen (durchgängig Apache-2.0/MIT-artig), daher lohnt vor endgültiger Festlegung eine kurze rechtliche Prüfung, ob die dynamische Einbindung als eigener Service (kein direktes Linken in eine größere Kompilationseinheit, da ohnehin als API-getrennter Microservice betrieben) mit euren Open-Source-Lizenzzielen für das Gesamtsystem vereinbar ist. Rein technisch ist die Trennung als eigenständiger Service (statt eingebundener Bibliothek im selben Prozess) unproblematisch, da alle Services ohnehin nur über APIs kommunizieren.
- Ersatzdarstellungen (2.4): konkrete Liste, welche Zielformate je Quellformat standardmäßig vorgehalten werden (fachliche Priorisierung, z. B. ob Video-Transkription von Anfang an oder erst als späterer Ausbauschritt kommt) sowie Wahl der konkreten Transkriptions-/Konvertierungsbibliotheken/-dienste.
- Umlaufmappen (2.3): Referenzverhalten ist konzeptionell entschieden (dynamische Referenz während der Bearbeitung, fixierter Abschluss-Snapshot bei Vorgangsende) - offen bleibt die genaue technische Umsetzung der "dynamischen Referenz" im Datenmodell (z. B. Foreign Key auf das Dokument mit Auflösung zur Laufzeit vs. Trigger-/Event-basierte Nachführung), sollte mit dem Datenbankschema zusammen entschieden werden.
- Erzwungene Objekt-Hierarchie (2.2a): offen, ob eine nachträgliche Änderung von `allowedParentTypes` an einer bereits produktiv genutzten Klasse rückwirkend gegen bestehende Ablagen geprüft wird (bestehende, nun "unzulässige" Platzierungen blockieren/melden) oder nur für künftige Neuanlagen/Verschiebungen gilt - fachliche Entscheidung, die eher mit den ersten Zielkunden/Klassenschemata getroffen werden sollte als abstrakt vorab.
- Klassen-Icons (2.2a/2.2b): genaue Herkunft/Format noch offen - kuratiertes Icon-Set zur Auswahl, freier SVG-Upload, oder beides (kuratiertes Set als Default, Upload als Erweiterung) - Sicherheitsprüfung (SVG kann aktiven Inhalt tragen) wäre bei freiem Upload

zusätzlich zu klären.

- Formular-Layouts (2.2b): das Grid-Modell (Zeilen/Spalten, Responsive-Breakpoint) ist konzeptionell entschieden, das genaue Datenformat des Layout-JSON (z. B. wie Feldbreiten pro Spalte, verschachtelte Gruppen oder bedingte Sichtbarkeit einzelner Felder abgebildet werden) ist Sache der jeweiligen Implementierungssession.
- Storage-Datenträger-Wechsel (3.6): das genaue Ablageformat/-verfahren der Identitätsdatei sowie die Priorisierung/Drosselung der Hintergrund-Replikation nach einem degradierten Start (z. B. gedrosselt während Kernarbeitszeit, volle Geschwindigkeit außerhalb) sind Implementierungsdetails der jeweiligen Session, keine Konzeptentscheidung.
- OCR-Konfiguration (3.9): konkrete Einheit/Schätzmethode der Wortobergrenze (z. B. anhand Seitenzahl/Bildauflösung geschätzt vs. exakte Vorab-Analyse) sowie sinnvolle Default-Batch-Size sind Implementierungsdetails, keine Konzeptentscheidung.